Computer Networks

Kiarash Sanei

June 2026

Contents

1 Computer Networks and the Internet **2**
1.1 The Internet: Two Views . . . 2
1.1.1 The “nuts-and-bolts” view . . . 2
1.1.2 The “services” view . . . 2
1.2 What Is a Protocol? . . . 2
1.3 The Network Edge: Hosts and Access Networks . . . 2
1.3.1 Physical media . . . 3
1.4 The Network Core: Packet vs. Circuit Switching . . . 3
1.4.1 Circuit switching . . . 3
1.4.2 Packet switching . . . 3
1.5 Two Key Network-Core Functions . . . 3
1.6 Sharing the Network: Statistical Multiplexing . . . 3
1.6.1 Why packet switching usually wins . . . 4
1.7 Performance: Delay, Loss, Throughput . . . 4
1.7.1 Four sources of nodal delay . . . 4
1.7.2 Traffic intensity and queueing . . . 4
1.7.3 Packet loss . . . 4
1.7.4 Throughput . . . 4
1.7.5 Traceroute . . . 4
1.8 Protocol Layers and the Internet Stack . . . 5
1.8.1 The five-layer Internet stack . . . 5
1.8.2 The narrow waist . . . 5
1.8.3 Encapsulation . . . 5
1.8.4 Who implements which layers . . . 5
1.8.5 Layer 3 is best-effort . . . 5
1.9 Network Security (Introduction) . . . 5
1.10 A Brief History . . . 6

2 The Application Layer **6**
2.1 Principles of Network Applications 6
2.1.1 Processes, sockets, and addressing 6
2.1.2 What an application-layer protocol defines 6
2.2 What Transport Service Does an App Need? 6
2.2.1 TCP vs. UDP at a glance 7
2.2.2 Securing TCP: TLS 7
2.3 Socket Programming (Completing the Roadmap) 7
2.3.1 UDP: connectionless 7
2.3.2 TCP: connection-oriented 7
2.4 Web and HTTP 8
2.4.1 Non-persistent vs. persistent HTTP 8
2.4.2 HTTP message format 8
2.4.3 Trying HTTP by hand 8
2.5 Maintaining State: Cookies 9
2.6 Web Caching (Proxy Servers) 9
2.6.1 Worked example 10
2.6.2 Conditional GET 10
2.7 HTTP/2 and HTTP/3 10
2.8 Email: SMTP and IMAP (Completing the Roadmap) 10
2.9 The Domain Name System (DNS) 11
2.9.1 Goals and why it's not centralized 11
2.9.2 The hierarchy 11
2.9.3 Iterative vs. recursive resolution 11
2.9.4 Packet format and records 11
2.9.5 Doing it yourself with `dig` 12
2.9.6 Other uses of DNS 12
2.9.7 DNS security 12
2.10 P2P and BitTorrent 12
2.10.1 File-distribution time: client–server vs. P2P 12
2.10.2 BitTorrent mechanics 12
2.11 Video Streaming and CDNs 13
2.11.1 Video basics 13
2.11.2 Playout buffering 13
2.11.3 DASH (Dynamic Adaptive Streaming over HTTP) 13
2.11.4 Content Distribution Networks 13

3 The Transport Layer **13**
3.1 Services and Principles 13
3.2 Multiplexing and Demultiplexing 14

3.2.1 Connectionless (UDP) demux . . . 14
3.2.2 Connection-oriented (TCP) demux . . . 14
3.3 UDP: User Datagram Protocol (RFC 768) . . . 14
3.3.1 UDP segment and checksum . . . 14
3.4 Principles of Reliable Data Transfer (rdt) . . . 14
3.4.1 Why stop-and-wait performs terribly . . . 15
3.5 Pipelined Protocols . . . 15
3.5.1 Go-Back-N (GBN) . . . 15
3.5.2 Selective Repeat (SR) . . . 15
3.6 TCP: Connection-Oriented Transport (RFCs 793, 1122, 2018, 5681, 7323) . . . 15
3.6.1 Segment structure . . . 15
3.6.2 Sequence numbers and ACKs . . . 15
3.6.3 RTT estimation and timeout . . . 16
3.6.4 Reliable transfer and fast retransmit . . . 16
3.7 TCP Flow Control . . . 16
3.8 TCP Connection Management . . . 16
3.8.1 Why 2-way handshakes fail . . . 16
3.8.2 The 3-way handshake . . . 16
3.8.3 Closing a connection . . . 17
3.9 Principles of Congestion Control . . . 17
3.10 TCP Congestion Control . . . 17
3.10.1 AIMD . . . 17
3.10.2 Sending rate and the window . . . 17
3.10.3 Slow start, congestion avoidance, fast recovery . . . 17
3.10.4 TCP throughput . . . 17
3.10.5 TCP CUBIC . . . 18
3.10.6 Delay-based congestion control (and BBR) . . . 18
3.10.7 ECN (Explicit Congestion Notification) . . . 18
3.10.8 TCP fairness . . . 18

4 The Network Layer: Data Plane 18
4.1 Overview: Data Plane vs. Control Plane . . . 18
4.1.1 Two key functions . . . 18
4.1.2 The two planes . . . 18
4.2 Network-Layer Service Models . . . 19
4.3 Inside a Router . . . 19
4.3.1 Input port functions . . . 19
4.4 Longest Prefix Matching (LPM) . . . 19
4.5 Switching Fabrics . . . 19

4.6 Input Queueing and HOL Blocking 20
4.7 Output Queueing and Buffering 20
4.7.1 How much buffering? 20
4.8 Buffer Management and Packet Scheduling 20
4.8.1 Buffer management 20
4.8.2 Scheduling policies 20
4.9 IP: The Internet Protocol — Datagram Format 20
4.10 IP Fragmentation and Reassembly 21
4.10.1 Worked example 21
4.11 IP Addressing, Subnets, and CIDR 21
4.11.1 Subnets 21
4.11.2 CIDR 22
4.12 Getting an Address: DHCP 22
4.12.1 Host part — DHCP 22
4.12.2 Network part — ISPs and ICANN 22
4.13 Hierarchical Addressing and Route Aggregation 22
4.14 NAT: Network Address Translation 22
4.14.1 How it works 23
4.15 IPv6 23
4.15.1 Datagram format 23
4.15.2 Transition: tunneling 23

5 The Network Layer: Control Plane 23
5.1 Overview: Two Approaches 23
5.2 Routing Protocols and the Graph Abstraction 24
5.3 Link-State Routing: Dijkstra's Algorithm 24
5.3.1 Worked example (source u) 24
5.4 Distance-Vector Routing: Bellman-Ford 25
5.4.1 Worked example 25
5.4.2 Convergence example (3 nodes) 25
5.4.3 Link-cost changes: good news vs. bad news 25
5.5 Making Routing Scalable: Autonomous Systems 26
5.6 Intra-AS Routing: RIP, EIGRP, OSPF 26
5.7 Inter-AS Routing: BGP 26
5.7.1 BGP sessions and messages 27
5.7.2 Path attributes 27
5.7.3 Path propagation and forwarding tables 27
5.7.4 Hot-potato routing and policy 27
5.7.5 Why intra- and inter-AS routing differ 27
5.8 The SDN Control Plane 27

5.9 ICMP: Internet Control Message Protocol . . . 28
5.9.1 Traceroute via ICMP . . . 28
5.10 Network Management: SNMP and NETCONF/YANG (Brief) . . . 28

6 The Link Layer and LANs 28
6.1 Introduction and Services . . . 28
6.2 Error Detection and Correction . . . 28
6.2.1 CRC . . . 29
6.3 Multiple Access Protocols . . . 29
6.3.1 Channel partitioning . . . 29
6.3.2 Random access . . . 29
6.3.3 Taking turns . . . 30
6.4 MAC Addressing and ARP . . . 30
6.4.1 MAC addresses . . . 30
6.4.2 ARP . . . 30
6.4.3 Routing to another subnet . . . 30
6.5 Ethernet . . . 31
6.5.1 Frame structure . . . 31
6.6 Switches . . . 31
6.6.1 Switch table and self-learning . . . 31
6.6.2 Switches vs. routers . . . 31
6.7 VLANs . . . 31
6.8 Link Virtualization and Data Centers (Brief) . . . 32
6.9 A Day in the Life of a Web Request (Synthesis) . . . 32

7 Cheat Sheet 32
7.1 Key Formulas . . . 33
7.2 Well-Known Ports . . . 34
7.3 HTTP Methods and Status Codes . . . 34
7.4 DNS Record Types . . . 34
7.5 TCP vs. UDP . . . 34
7.6 Circuit vs. Packet Switching . . . 35
7.7 Go-Back-N vs. Selective Repeat . . . 35
7.8 The Five-Layer Stack (with units and example protocols) . . . 35
7.9 TCP Congestion-Control State Reactions . . . 35
7.10 Router Data Plane vs. Control Plane . . . 35
7.11 Packet Scheduling Policies . . . 36
7.12 IPv4 Header Fields (Quick Reference) . . . 36
7.13 CIDR / Subnet Mask Quick Reference . . . 36
7.14 Private and Special IPv4 Addresses . . . 36
7.15 IPv4 vs. IPv6 . . . 37

7.16 DHCP (DORA) and NAT 37
7.17 Routing Algorithms and Protocols 37
7.18 ICMP Common Messages 37
7.19 Multiple Access Protocols 38
7.20 Error Detection Methods 38
7.21 Link Layer: MAC vs. IP, Switch vs. Router 38
7.22 Ethernet Frame Fields 38
7.23 Intra-AS Routing Protocols 39
7.24 BGP: Attributes and Route Selection 39
7.25 ARP and Switch Self-Learning 39
7.26 VLANs and 802.1Q 39

1 Computer Networks and the Internet

1.1 The Internet: Two Views

1.1.1 The "nuts-and-bolts" view

The Internet is a vast interconnection of computing devices and the infrastructure that links them. It has three structural pieces:

- **Hosts (end systems):** billions of connected devices running network applications at the Internet's "edge" — laptops, phones, servers, but also cars, sensors, pacemakers, and "smart" appliances (IoT).
- **Packet switches:** devices that forward *packets* (chunks of data). The two main kinds are **routers** (network core) and **link-layer switches** (access/LAN).
- **Communication links:** fiber, copper, coaxial cable, radio, and satellite. Each link has a **transmission rate** (bandwidth) measured in bits/sec.

A **network** is a collection of devices, routers, and links managed by a single organization. The Internet is therefore best described as a **"network of networks"** — a global mesh of interconnected ISPs (Internet Service Providers).

1.1.2 The "services" view

Alternatively, the Internet is *infrastructure that provides services to distributed applications*: the Web, video streaming, email, games, e-commerce, social media, and so on. It provides a **programming interface** (the socket API) that lets applications "hook into" the Internet's transport service — much like a postal service offers different delivery options.

1.2 What Is a Protocol?

A **protocol** defines the *format* and *order* of messages exchanged between communicating entities, plus the *actions* taken on message transmission and receipt. Humans use protocols too ("Hi" → "Hi" → "Got the time?" → "2:00"); on the Internet, *all* communication activity is governed by protocols, just between machines instead of people.

- Examples span every layer: HTTP (Web), TCP, IP, WiFi, 4/5G, Ethernet, and many more.
- Internet standards are published as **RFCs** (Request for Comments) by the **IETF** (Internet Engineering Task Force).

1.3 The Network Edge: Hosts and Access Networks

Hosts are split into **clients** (request service) and **servers** (provide service, often housed in data centers). The **access network** physically connects an end system to its first ("edge") router.

- **Cable (HFC — hybrid fiber coax):** shares a coaxial distribution network among homes; uses **frequency-division multiplexing (FDM)** so data and TV travel in different frequency bands. Asymmetric: up to roughly 40 Mbps–1.2 Gbps downstream, 30–100 Mbps upstream. The network device at the head end is the CMTS (Cable Modem Termination System).
- **DSL (Digital Subscriber Line):** uses the existing telephone line to the central office (DSLAM). *Dedicated* (not shared) line: roughly 24–52 Mbps down, 3.5–16 Mbps up. Voice and data ride different frequencies on the same wire.
- **Home networks:** a single box typically combines cable/DSL modem, router, NAT, firewall, wired Ethernet (1 Gbps), and a WiFi access point (54/450 Mbps).
- **Enterprise networks:** a mix of Ethernet (100 Mbps / 1 / 10 Gbps) and WiFi connecting switches and routers.
- **Wireless access:** WLAN/WiFi (802.11, ∼10s of meters) vs. wide-area cellular 4G/5G (operator-provided, ∼10s of km).
- **Data-center networks:** high-bandwidth links (10s–100s Gbps) connecting thousands of servers.

1.3.1 Physical media

Guided media carry signals in a solid: twisted pair (Cat 5/6 copper), coaxial cable, fiber optic (glass, light pulses; 10s–100s Gbps, very low error rate, immune to electromagnetic noise). **Unguided** media let signals propagate freely: radio. Radio link types include WiFi, cellular, Bluetooth, terrestrial microwave, and satellite (geostationary satellites add ∼270 ms end-to-end delay).

1.4 The Network Core: Packet vs. Circuit Switching

The core is a mesh of interconnected routers. There are two fundamental ways to share core links among many application-level conversations.

1.4.1 Circuit switching

Resources (a "circuit") are **reserved** end-to-end for the duration of a call.

1. **Setup:** establish a circuit (e.g. when you dial a phone number); state is created in each switch along the path.
2. **Communicate:** send data over the dedicated, guaranteed circuit.
3. **Teardown:** release the circuit and remove the state.

Resources are dedicated (no sharing ⇒ guaranteed performance), but a circuit sits *idle* if the call has nothing to send. Two multiplexing techniques:

- **FDM:** the frequency spectrum is divided into bands; each call gets its own band.
- **TDM:** time is divided into frames, each frame into slots; each call gets a periodic slot and transmits at the full link rate *during* its slot. Slot position within the frame identifies the conversation.

1.4.2 Packet switching

Used in the Internet. Messages are broken into **packets**, each carrying a header (control info such as source/destination addresses).

- **Store-and-forward:** a router must receive the *entire* packet before it can begin transmitting it onto the next link.
- **No reserved capacity:** each packet is forwarded independently using the router's **forwarding table** (keyed on the destination address in the header).
- **Queues:** each output link has a buffer/queue; packets wait their turn. If a queue is full, arriving packets are **dropped** (lost).

1.5 Two Key Network-Core Functions

- **Forwarding ("switching"):** a *local* action — move an arriving packet from an input link to the correct output link, consulting the local forwarding table.
- **Routing:** a *global* action — routing algorithms determine the end-to-end source–destination paths that packets take.

1.6 Sharing the Network: Statistical Multiplexing

Many *flows* (streams of packets between two hosts) must share routers and links. Two ways to allocate resources:

- **Static allocation:** give each user a fixed share. Must provision for everyone's *peak* demand ⇒ wasteful.
- **Statistical multiplexing:** allocate dynamically, on demand. More efficient because

$$\text{peak of aggregate demand} \;<\; \text{aggregate of peak demands.}$$

Example. Alice needs 10 units in the morning and 2 the rest of the time; Bob needs 10 at night and 2 otherwise. Static: each gets 10 always ⇒ provision 20. Dynamic: hand out 10 to whoever peaks and 2 to the other ⇒ provision only 12.

The two canonical ways to implement multiplexing map directly onto the two switching styles:

- **Reservations via circuit switching:** explicitly reserve, use, then release resources (the "restaurant reservation" analogy).
- **Best-effort via packet switching:** just send packets and "hope for the best" ("first-come, first-served").

1.6.1 Why packet switching usually wins

Consider a 1 Gbps link with users that each need 100 Mbps when active, and are active only 10% of the time.

- **Circuit switching:** can support only $1000/100 = 10$ users (each statically reserved).
- **Packet switching:** with 35 users, the probability that more than 10 are simultaneously active is < 0.0004, so the link almost never overflows.

The per-user active count follows a **binomial distribution**: with n independent users each active with probability p,

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad P(X > 10) = \sum_{k=11}^{n} \binom{n}{k} p^k (1-p)^{n-k}.$$

Packet switching is **great for bursty data** (resource sharing, no call setup) but allows **congestion** (delay and loss from buffer overflow), so it needs protocols for reliable transfer and congestion control.

1.7 Performance: Delay, Loss, Throughput

1.7.1 Four sources of nodal delay

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

- d_{proc} — **processing**: check bit errors, determine output link. Typically $<$ microseconds.
- d_{queue} — **queueing**: time waiting at the output link; depends on congestion.
- d_{trans} — **transmission**: time to push all L bits onto the link at rate R: $d_{\text{trans}} = L/R$.
- d_{prop} — **propagation**: time for a bit to travel the physical link of length d at speed $s \approx 2 \times 10^8$ m/s: $d_{\text{prop}} = d/s$.

Transmission vs. propagation are different things. The caravan analogy: cars = bits, a caravan = a packet, a toll booth = a link/router. "Pushing" the whole caravan through a booth is transmission delay; a car driving between booths is propagation delay.

1.7.2 Traffic intensity and queueing

With average packet arrival rate a, packet length L, and link rate R, the **traffic intensity** is La/R:

$$\frac{La}{R} \approx 0 \;\Rightarrow\; \text{small avg. queueing delay}, \quad \frac{La}{R} \to 1 \;\Rightarrow\; \text{large delay}, \quad \frac{La}{R} > 1 \;\Rightarrow\; \text{delay} \to \infty.$$

Rule of thumb: design so that traffic intensity stays comfortably below 1.

1.7.3 Packet loss

Router buffers are finite. When a packet arrives to a *full* buffer it is dropped. A lost packet may be retransmitted by the previous node, by the source end system, or not at all.

1.7.4 Throughput

Throughput is the rate (bits/time) at which bits actually reach the receiver (instantaneous vs. average). On a path with link rates R_s (server side) and R_c (client side), end-to-end throughput is set by the **bottleneck link**:

$$\text{throughput} = \min(R_s, R_c).$$

If N connections fairly share a backbone link of rate R, the per-connection throughput is

$$\min\left(R_c,\ R_s,\ \frac{R}{N}\right),$$

and in practice R_c or R_s (the access links) is usually the bottleneck.

1.7.5 Traceroute

`traceroute` measures delay to each router on the path: it sends three probes with increasing TTL ($= i$) so probe i expires at router i, which returns an error; the sender times the round trip. `*` means no reply.

1.8 Protocol Layers and the Internet Stack

Networks are complex (hosts, routers, many media, applications). Layering imposes structure and **modularity**: each layer relies on services from the layer below and provides services to the layer above. A change inside one layer (e.g. swapping a Layer-2 protocol) is transparent to the others — just as changing the airline gate procedure doesn't affect baggage handling.

1.8.1 The five-layer Internet stack

- **Application** — network applications and their protocols: HTTP, IMAP, SMTP, DNS. Unit: *message*.
- **Transport** — process-to-process data transfer: TCP, UDP. Unit: *segment*.
- **Network** — routing of datagrams source-to-destination: IP and routing protocols. Unit: *datagram*.
- **Link** — data transfer between neighboring nodes: Ethernet, 802.11 (WiFi), PPP. Unit: *frame*.
- **Physical** — bits "on the wire."

(The OSI model adds *presentation* and *session* layers, 6 and 5, now effectively obsolete on the Internet.)

1.8.2 The narrow waist

IP is the only protocol at the network layer — every host and router speaks IP. This single "narrow waist" unifies the Internet and lets arbitrary link-layer technologies below and arbitrary applications above interoperate (the "hourglass").

1.8.3 Encapsulation

As a message descends the stack, each layer wraps the unit from above in its own header (Matryoshka-doll style):

$$\underbrace{M}_{\text{message}} \rightarrow \underbrace{[H_t \,|\, M]}_{\text{segment}} \rightarrow \underbrace{[H_n \,|\, H_t \,|\, M]}_{\text{datagram}} \rightarrow \underbrace{[H_l \,|\, H_n \,|\, H_t \,|\, M]}_{\text{frame}}.$$

At the destination, headers are *peeled* off going up. Key insight from the "postal/CEO secretary" analogy: **each layer only reads the header written by its peer at the same layer.** Layer-3 (IP) destination is always the final endpoint; the Layer-2 header changes hop-by-hop.

1.8.4 Who implements which layers

- **End hosts** implement *all* layers (1–7). In hardware (the NIC): layers 1–2. In the OS: layers 3–4. In applications: layer 7.
- **Routers** implement only layers 1–3: they parse and forward, rewriting the Layer-2 header for the next hop, but do not provide reliable delivery (4) or touch application data (7).

1.8.5 Layer 3 is best-effort

The network layer offers a **best-effort** service model: packets are size-limited and may be lost, reordered, or corrupted; the network "tries its best" but gives no guarantee and won't even tell you when delivery fails. Higher layers (transport) must add reliability if the application needs it.

1.9 Network Security (Introduction)

The Internet was originally designed for "a group of mutually trusting users attached to a transparent network," so security has always been retrofitted.

- **Packet sniffing:** a promiscuous NIC on a broadcast medium (shared Ethernet, WiFi) records all passing packets — including cleartext passwords. (Wireshark is a benign sniffer used in labs.)
- **IP spoofing:** inject packets with a forged source address.
- **Denial of Service (DoS):** overwhelm a target (server, bandwidth) with bogus traffic, often from a *botnet* of compromised hosts (DDoS).

Lines of defense: authentication (e.g. SIM hardware identity), confidentiality (encryption), integrity (digital signatures), access restrictions (VPNs, passwords), and **firewalls** ("off-by-default" middleboxes that filter traffic and react to DoS).

1.10 A Brief History

- **1950s–60s, Cold War:** the US wanted a communications system that could survive a nuclear attack. **ARPA** (1958, a response to Sputnik; later DARPA) funded large-scale networking research.
- **Paul Baran (RAND, early 1960s):** survivable comms via *decentralized architecture* (no single point of failure), *message blocks* (packets sent independently and reassembled), and *hot-potato routing* (forward as fast as possible).
- **Donald Davies (UK):** coined the term "packet." **Leonard Kleinrock (MIT, 1961–62):** packet-switching theory. **Licklider:** vision of a "galactic network."
- **Kleinrock → Roberts:** Roberts wrote the ARPANET plan (1967); DARPA's 1968 RFQ for the packet switches (Interface Message Processors, IMPs) was won by **BBN**.
- **1969:** first nodes — UCLA (Kleinrock's Network Measurement Center) and SRI. The first host-to-host message ("LO…" then the system crashed) was sent UCLA→SRI; UCSB and Utah followed.
- **1970:** the Network Working Group (Crocker) finished NCP, the first host-to-host protocol. **1972:** Kahn demoed ARPANET at ICCC; Ray Tomlinson (BBN) wrote email.
- **Open-architecture / internetworking:** Kahn's four ground rules — (1) each network stands on its own, (2) best-effort delivery with source retransmission, (3) simple stateless "black boxes" (gateways/routers), (4) no global control. Kahn & Cerf's 1974 paper defined **TCP/IP**, later split into IP (addressing/forwarding), TCP (reliability), and UDP (direct access to IP).
- **Documentation:** Crocker started the **RFC** series (1969); the **IETF** (founded 1986) makes voluntary standards published as RFCs.

2 The Application Layer

2.1 Principles of Network Applications

You write applications that run on *end systems* and communicate over the network; you never need to write software for core devices (they don't run user applications). Two dominant architectures:

- **Client–server:** an always-on server with a permanent IP address (often in a data center for scaling) serves intermittently connected clients. Clients don't talk to each other directly. Examples: HTTP, IMAP, FTP.
- **Peer-to-peer (P2P):** no always-on server; peers request and provide service to each other. *Self-scaling* (each new peer adds capacity *and* demand) but harder to manage (peers churn and change IPs). Example: BitTorrent.

2.1.1 Processes, sockets, and addressing

A **process** is a running program. Processes on different hosts communicate by exchanging messages through **sockets** — the "door" between the application and the transport layer. To be addressable, a process needs an **IP address** (identifies the host) *and* a **port number** (identifies the process/socket on that host), because many processes run on one host. Example well-known ports: HTTP = 80, SMTP = 25, DNS = 53.

2.1.2 What an application-layer protocol defines

The types of messages (request/response), message *syntax* (fields and how they're delineated), message *semantics* (meaning of fields), and *rules* for when/how to send and respond. **Open** protocols are defined in RFCs (HTTP, SMTP) and enable interoperability; **proprietary** protocols are closed (e.g. some video-conferencing codecs).

2.2 What Transport Service Does an App Need?

Apps care about four dimensions:

- **Data integrity:** file transfer needs 100% reliability; audio tolerates some loss.
- **Timing:** interactive games / telephony need low delay.
- **Throughput:** multimedia needs a minimum rate; "elastic" apps use whatever they get.
- **Security:** encryption, integrity, authentication.

2.2.1 TCP vs. UDP at a glance

- **TCP:** reliable, in-order delivery; flow control; congestion control; connection setup. Does *not* guarantee timing or minimum throughput.
- **UDP:** unreliable, unordered; no setup, no flow/congestion control. Minimal overhead. "Why bother?" — it's fast and works under congestion, and apps can add their own reliability on top (HTTP/3 does exactly this).

2.2.2 Securing TCP: TLS

Vanilla TCP/UDP sockets send cleartext. **TLS** (Transport Layer Security) is implemented in the *application layer* (apps link a TLS library that uses TCP underneath); it provides encrypted connections, data integrity, and endpoint authentication.

2.3 Socket Programming (Completing the Roadmap)

The slides list socket programming with UDP and TCP but don't show code. Here are minimal Python examples that make the client–server paradigm concrete.

2.3.1 UDP: connectionless

There is no handshake; every datagram is self-contained and must carry the destination address.

```python
# --- UDP server ---
from socket import socket, AF_INET, SOCK_DGRAM

server_port = 12000
s = socket(AF_INET, SOCK_DGRAM) # SOCK_DGRAM = UDP
s.bind(('', server_port)) # listen on all interfaces
print("UDP server ready")

while True:
    message, client_addr = s.recvfrom(2048) # blocks for a datagram
    reply = message.decode().upper()
    s.sendto(reply.encode(), client_addr) # must specify dest each time

# --- UDP client ---
from socket import socket, AF_INET, SOCK_DGRAM

server_name, server_port = 'localhost', 12000
s = socket(AF_INET, SOCK_DGRAM)
msg = input("Input lowercase sentence: ")
s.sendto(msg.encode(), (server_name, server_port))
reply, _ = s.recvfrom(2048)
print(reply.decode())
s.close()
```

Listing 1: UDP echo server and client

2.3.2 TCP: connection-oriented

The server has a *welcoming* socket; `accept()` returns a *new* connection socket dedicated to that one client (this is the 4-tuple demultiplexing we'll see in the transport layer).

```python
# --- TCP server ---
from socket import socket, AF_INET, SOCK_STREAM

server_port = 12000
welcome = socket(AF_INET, SOCK_STREAM) # SOCK_STREAM = TCP
welcome.bind(('', server_port))
welcome.listen(1) # backlog of pending connections
print("TCP server ready")

while True:
    conn, addr = welcome.accept() # new socket per client
```

```
    sentence = conn.recv(1024).decode()
    conn.send(sentence.upper().encode())
    conn.close()

# --- TCP client ---
from socket import socket, AF_INET, SOCK_STREAM

server_name, server_port = 'localhost', 12000
s = socket(AF_INET, SOCK_STREAM)
s.connect((server_name, server_port)) # triggers the 3-way handshake
s.send(input("Input sentence: ").encode())
print(s.recv(1024).decode())
s.close()
```

Listing 2: TCP echo server and client

2.4 Web and HTTP

A **web page** consists of objects (an HTML base file plus referenced objects — JPEGs, scripts, etc.), each addressable by a **URL** of the form `hostname/pathname`. **HTTP** (HyperText Transfer Protocol) is the Web's application-layer protocol, developed by Tim Berners-Lee at CERN in 1989 (HTTP/0.9 in 1991, HTTP/1.0 in 1996, HTTP/1.1 in 1997).

- **Client–server, over TCP:** client opens a TCP connection to port 80, exchanges request/response messages, then the connection closes.
- **Stateless:** the server keeps *no* memory of past client requests. Stateful protocols are complex — after a crash, the two sides' views of "state" can diverge and must be reconciled.

2.4.1 Non-persistent vs. persistent HTTP

- **Non-persistent (HTTP/1.0):** at most one object per TCP connection. Each object costs roughly $2\,RTT$ (one RTT to set up TCP, one for the request + first bytes) plus the file's transmission time.

$$T_{\text{non-persistent (per object)}} = 2\,RTT + T_{\text{transmit}}.$$

- **Persistent (HTTP/1.1):** the server leaves the connection open; multiple objects reuse it. As little as one RTT for all referenced objects.

2.4.2 HTTP message format

Requests are ASCII (human-readable). A request line (method, URL, version) is followed by header lines; methods include `GET`, `POST` (form data in the entity body), `HEAD` (headers only), and `PUT` (upload/replace).

```
GET /index.html HTTP/1.1
Host: www-net.cs.umass.edu
User-Agent: Mozilla/5.0 (...) Firefox/80.0
Accept: text/html,application/xhtml+xml
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive
                      <-- a blank line (just CRLF) ends the headers
```

Listing 3: A raw HTTP/1.1 GET request (CRLF = \r\n)

A response begins with a **status line** (version, status code, phrase), then headers, then the data. Common status codes: **200** OK, **301** Moved Permanently (new URL in `Location:`), **400** Bad Request, **404** Not Found, **505** HTTP Version Not Supported.

2.4.3 Trying HTTP by hand

You can speak HTTP manually with `netcat`/`ncat`:

```
# Open a TCP connection to port 80, then type the request:
nc -c -v gaia.cs.umass.edu 80 # macOS
ncat -C gaia.cs.umass.edu 80 # Windows

GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
                                # press Enter twice (blank line)
```

Listing 4: Manual HTTP request over a raw TCP connection

A more typical programmatic fetch:

```
from socket import socket, AF_INET, SOCK_STREAM

host = 'gaia.cs.umass.edu'
s = socket(AF_INET, SOCK_STREAM)
s.connect((host, 80))
request = (f"GET / HTTP/1.1\r\n"
          f"Host: {host}\r\n"
          f"Connection: close\r\n\r\n") # blank line terminates headers
s.send(request.encode())

response = b""
while True:
    chunk = s.recv(4096)
    if not chunk:
        break
    response += chunk
s.close()
print(response.decode(errors='replace')[:500])
```

Listing 5: A raw HTTP GET via a socket

2.5 Maintaining State: Cookies

HTTP itself is stateless, but sites keep state with **cookies**, which have four components:

1. A `Set-Cookie:` header in the HTTP *response*.
2. A `Cookie:` header in subsequent HTTP *requests*.
3. A cookie file on the user's host, managed by the browser.
4. A back-end database at the site.

On a first visit the site creates a unique ID and a database entry; every later request carries the ID so the site can "recognize" the user.

- **Uses:** authorization, shopping carts, recommendations, web-email session state.
- **First-party** cookie: from the site you chose to visit. **Third-party / tracking** cookie: from an ad/tracker (e.g. `AdX.com`) embedded across many sites, letting a common ID follow you around — often via an invisible link rather than a visible ad. Increasingly disabled by default and regulated (e.g. EU GDPR treats identifying cookies as personal data).

2.6 Web Caching (Proxy Servers)

A **web cache** sits between clients and origin servers, acting as both server (to the client) and client (to the origin). The browser sends all requests to the cache; on a hit it returns the object directly, on a miss it fetches from the origin, caches a copy, and returns it.

Why cache? Reduce client response time (cache is nearer), reduce traffic on the institution's access link, and let "poor" content providers deliver effectively.

2.6.1 Worked example

Origin RTT = 2 s, object size = 100 Kbits, request rate = 15/s, access link = 1.54 Mbps, LAN = 1 Gbps. Average data rate to browsers = 15×100 Kbits = 1.50 Mbps.

- **No cache:** access-link utilization = $1.50/1.54 \approx 0.97 \Rightarrow$ huge queueing delay (end-to-end delay = 2 s + *minutes* $+\mu$s).
- **Faster link (154 Mbps):** utilization ≈ 0.0097, delay in msec — but expensive.
- **Install a cache (hit rate** 0.4**):** only 60% of traffic hits the access link $\Rightarrow 0.6 \times 1.50 = 0.9$ Mbps, utilization $0.9/1.54 \approx 0.58 \Rightarrow$ low delay. Average end-to-end delay $\approx 0.6(2.01) + 0.4(\sim\text{msec}) \approx 1.2$ s — lower *and* cheaper than the fast link.

2.6.2 Conditional GET

To avoid sending an object the browser already has fresh, the client adds `If-Modified-Since: <date>`. The server replies `304 Not Modified` (no body) if unchanged, or `200 OK` with the object if it changed.

2.7 HTTP/2 and HTTP/3

HTTP/2 (RFC 7540, 2015) targets reduced delay for multi-object pages. HTTP/1.1 introduced pipelined GETs over one TCP connection, but the server replies FCFS, so a small object can suffer **head-of-line (HOL) blocking** behind a large one, and TCP loss recovery stalls everything. HTTP/2 keeps methods/status codes/most headers but adds:

- client-specified object **priority** (not necessarily FCFS),
- server **push** of unrequested objects,
- dividing objects into **frames** and *interleaving* them to mitigate HOL blocking.

HTTP/3 addresses HTTP/2's remaining problem: a single TCP connection means one packet loss still stalls all streams. HTTP/3 runs over **UDP** (via QUIC), adding security plus *per-object* error and congestion control.

2.8 Email: SMTP and IMAP (Completing the Roadmap)

The chapter roadmap lists "E-mail, SMTP, IMAP" but the slides skip it. Here is the essential picture.

- **Components:** user agents (mail clients), mail servers (each with an outgoing *message queue* and per-user *mailbox*), and the **SMTP** protocol between servers.
- **SMTP (Simple Mail Transfer Protocol, RFC 5321):** a *push* protocol over TCP **port 25**; uses persistent connections; commands and replies are 7-bit ASCII; a message ends with a line containing only ".".
- **Retrieval:** SMTP delivers mail *to* the recipient's server, but a user agent *pulls* mail from its mailbox with **IMAP** (keeps messages on the server, supports folders/search) or the older POP3. Webmail uses HTTP for this last hop.

```
telnet mail.example.com 25
S: 220 mail.example.com ESMTP ready
C: HELO relay.local
S: 250 Hello relay.local
C: MAIL FROM:<alice@example.com>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CRLF>.<CRLF>
C: Subject: Hi Bob
C:
C: Your days are numbered.
C: .
S: 250 OK queued
C: QUIT
S: 221 Bye
```

Listing 6: A bare SMTP dialog (client lines after the prompts)

2.9 The Domain Name System (DNS)

DNS translates human-readable domain names (`www.google.com`) into IP addresses (`74.125.25.99`). It is itself an *application-layer* protocol, even though name resolution is a core Internet function — complexity is pushed to the network's edge.

2.9.1 Goals and why it's not centralized

DNS must be **scalable** (billions of records, trillions of queries/day), **highly available** (no single point of failure), and **fast** (almost every connection begins with a lookup, so a slow DNS makes everything slow). Centralizing would fail on all of these (single point of failure, traffic volume, distance, maintenance). It is read-heavy: far more lookups than updates.

2.9.2 The hierarchy

DNS arranges **name servers** in a tree; if one can't answer, it *delegates* to a child.

- **Root servers** (responsible for everything; only 13 root-server *names*, `a`–`m.root-servers.net`, but thousands of physical servers via **anycast**).
- **Top-Level Domain (TLD) servers:** `.com`, `.org`, `.edu`, country domains, etc. (Verisign/Network Solutions for `.com`; Educause for `.edu`). Over 1500 TLDs today.
- **Authoritative servers:** an organization's own servers with the definitive records for its hosts.

Three kinds of hierarchy: names (dotted), authority (a *zone* = an administrative authority that can delegate sub-zones), and infrastructure (each server knows only a subset).

2.9.3 Iterative vs. recursive resolution

- **Iterative:** the contacted server replies "I don't know, but ask *that* server." The client walks the tree itself.
- **Recursive:** the client offloads the whole job to its **recursive resolver** (e.g. ISP-run, or `8.8.8.8` /Google, `1.1.1.1`/Cloudflare). The resolver does the iterative queries and returns the final answer. Resolvers build large shared caches and reduce load on name servers. Your machine's **stub resolver** just asks the recursive resolver.

Answers are **cached** for a **TTL** (e.g. 86 400 s = 24 h).

2.9.4 Packet format and records

DNS runs over **UDP port 53** (lightweight, no handshake, no per-connection state; usually fits one packet). Large transfers (e.g. zone transfers to secondary servers) fall back to TCP. The header carries a 16-bit **ID** (matches responses to queries; should be random for security), **flags** (QR query/response bit; RD recursion-desired bit), and four record **counts**. The payload holds **resource records (RRs)** — name–value pairs sorted into Question / Answer / Authority / Additional sections.

Record types you should know:

- **A** — name → IPv4 address.
- **AAAA** — name → IPv6 address.
- **NS** — delegates a domain to another name server.
- **CNAME** — alias for a canonical name.
- **MX** — mail server for a domain (with a priority; try lowest number first).

2.9.5 Doing it yourself with dig

```
# Ask the root directly; it delegates to .edu
$ dig +norecurse eecs.berkeley.edu @198.41.0.4
;; AUTHORITY SECTION:
edu. 172800 IN NS a.edu-servers.net.
;; ADDITIONAL SECTION:
a.edu-servers.net. 172800 IN A 192.5.6.30 # glue record

# Now ask .edu; it delegates to berkeley.edu
$ dig +norecurse eecs.berkeley.edu @192.5.6.30

# Finally ask berkeley.edu; it answers authoritatively
$ dig +norecurse eecs.berkeley.edu @128.32.136.3
;; ANSWER SECTION:
eecs.berkeley.edu. 86400 IN A 23.185.0.1
```

Listing 7: Walking the hierarchy manually with +norecurse

```
import socket
print(socket.gethostbyname('www.google.com')) # -> e.g. 142.250.x.x
# Modern, supports IPv6 and more:
for fam, *_ , sockaddr in socket.getaddrinfo('example.com', 443):
    print(sockaddr[0])
```

Listing 8: Resolving names from a program

2.9.6 Other uses of DNS

- **Email** via MX records.
- **Load balancing:** return multiple A records (client picks one; server shuffles order) for coarse-grained balancing and resiliency.
- **Geographic load balancing:** return different answers based on the (resolver's) location to steer clients to a "nearby" server — inherently approximate.

2.9.7 DNS security

DDoS on root servers (mitigated by traffic filtering and resolver caching of TLD addresses) or the more dangerous TLD servers; **spoofing / cache poisoning** (returning bogus replies). **DNSSEC** (RFC 4033) adds authentication.

2.10 P2P and BitTorrent

2.10.1 File-distribution time: client–server vs. P2P

Distributing a file of size F to N peers; server upload u_s, peer i upload/download u_i, d_i, with $d_{\min} = \min_i d_i$.

$$D_{\text{c-s}} \;\geq\; \max\left\{\frac{NF}{u_s},\; \frac{F}{d_{\min}}\right\} \qquad \text{(grows } \textit{linearly} \text{ in } N\text{)}$$

$$D_{\text{P2P}} \;\geq\; \max\left\{\frac{F}{u_s},\; \frac{F}{d_{\min}},\; \frac{NF}{u_s + \sum_i u_i}\right\}$$

In P2P, demand NF *and* supply $u_s + \sum_i u_i$ both grow with N, so distribution time scales far better — the source of P2P's **self-scalability**.

2.10.2 BitTorrent mechanics

- A file is split into **256-Kb chunks**; a **torrent** is the group of peers exchanging chunks; a **tracker** tracks participating peers.
- A new peer registers with the tracker, gets a list of peers, connects to a subset ("neighbors"), and accumulates chunks while uploading what it has. **Churn:** peers join/leave; a finished peer may leave (selfish) or seed (altruistic).

- **Requesting — rarest first:** Alice periodically asks neighbors for their chunk lists and requests the *rarest* missing chunks first.
- **Sending — tit-for-tat:** Alice sends to the four peers feeding her fastest (others are "choked"); re-evaluated every 10 s. Every 30 s she "optimistically unchokes" a random peer to discover better partners. Higher upload rate ⇒ better trading partners ⇒ faster downloads.

2.11 Video Streaming and CDNs

Streaming video (Netflix, YouTube, Amazon Prime) was ∼80% of residential ISP traffic (2020). Challenges: **scale** (∼1B users) and **heterogeneity** (wired vs. mobile, rich vs. poor bandwidth).

2.11.1 Video basics

A video is a sequence of images (e.g. 24–30 fps); each image is an array of pixels. **Coding** exploits redundancy to cut bits: *spatial* (within an image — e.g. run-length encode a block of identical color) and *temporal* (between frames — send only the difference from the previous frame). **CBR** (constant bit rate) vs. **VBR** (variable bit rate). Examples: MPEG-1 1.5 Mbps, MPEG-2 3–6 Mbps, MPEG-4 64 Kbps–12 Mbps.

2.11.2 Playout buffering

Network delay is variable (**jitter**), but playout must be continuous. A **client-side buffer** absorbs jitter: the client pre-fills the buffer and plays out at a constant rate while reception varies, adding a **playout delay**.

2.11.3 DASH (Dynamic Adaptive Streaming over HTTP)

- **Server:** divides the video into *chunks*, encodes each at multiple rates stored in separate files, and provides a **manifest** listing chunk URLs; files are replicated across CDN nodes.
- **Client (the "intelligence"):** periodically estimates available bandwidth and, consulting the manifest, requests one chunk at a time — choosing *when* (avoid buffer starvation/overflow), *what rate* (highest sustainable quality), and *where* (a CDN node that is close or uncongested).

2.11.4 Content Distribution Networks

A single "megaserver" doesn't scale (single point of failure, congestion, long paths). A **CDN** stores copies at many geographically distributed sites. Two philosophies:

- **Enter deep** (e.g. Akamai, ∼240k servers in >120 countries): push servers *deep* into access networks, close to users.
- **Bring home** (e.g. Limelight): fewer, larger clusters in points-of-presence (POPs) near access nets.

Netflix OpenConnect stores content at CDN nodes; the client gets a manifest and pulls each chunk at the highest supportable rate (or a different copy if a path is congested). Core **OTT** ("over the top") challenges: *which* content goes in *which* node, and *from where / at what rate* to retrieve it.

3 The Transport Layer

3.1 Services and Principles

The transport layer provides **logical end-to-end communication between processes**, building on the network layer's host-to-host service.

- **Sender:** breaks application messages into **segments**, passes them to the network layer.
- **Receiver:** reassembles segments into messages, passes them up to the application.

Household analogy: 12 kids in Ann's house writing to 12 kids in Bill's house. Houses = hosts, kids = processes, letters = messages, the postal service = network layer, and *Ann and Bill* (who sort mail to the right sibling) = the transport protocol.

3.2 Multiplexing and Demultiplexing

- **Multiplexing (sender):** gather data from multiple sockets, add transport headers.
- **Demultiplexing (receiver):** use header fields to deliver each segment to the correct socket.

3.2.1 Connectionless (UDP) demux

A UDP socket is identified by the **2-tuple** (*dest IP, dest port*). Two datagrams with the same destination port but different source IPs/ports are delivered to the *same* socket.

3.2.2 Connection-oriented (TCP) demux

A TCP socket is identified by the full **4-tuple** (*src IP, src port, dest IP, dest port*). A server therefore supports many simultaneous sockets, one per connecting client — which is exactly why `accept()` returns a fresh socket. Three segments all sent to (B, 80) but from different sources demux to different sockets.

3.3 UDP: User Datagram Protocol (RFC 768)

"No frills, bare bones, best effort." Segments may be lost or delivered out of order. Benefits: **no connection setup** (no RTT penalty), *no* connection state, **small header**, *no* congestion control (can blast as fast as desired; functions under congestion). **Uses:** DNS, SNMP, streaming multimedia (loss-tolerant, rate-sensitive), and HTTP/3 (which re-adds reliability and congestion control at the application layer).

3.3.1 UDP segment and checksum

Header (8 bytes): source port, dest port, length (bytes, incl. header), checksum. The **Internet checksum** detects flipped bits:

- **Sender:** treat the segment (plus a pseudo-header with IP addresses) as a sequence of 16-bit integers; compute the one's-complement sum (a carry out of the top bit is added back — "wraparound"); store the one's complement of that sum in the checksum field.
- **Receiver:** sum all 16-bit words *including* the checksum; the result should be all ones. Not equal ⇒ error detected. (It's **weak** protection: compensating bit flips can leave the checksum unchanged.)

```python
def internet_checksum(data: bytes) -> int:
    if len(data) % 2: # pad to an even number of bytes
        data += b'\x00'
    total = 0
    for i in range(0, len(data), 2):
        word = (data[i] << 8) + data[i + 1]
        total += word
        total = (total & 0xFFFF) + (total >> 16) # fold the carry (wraparound)
    return ~total & 0xFFFF # one's complement

# Verify: summing data + checksum should give 0xFFFF (all ones)
```

Listing 9: The 16-bit Internet checksum

3.4 Principles of Reliable Data Transfer (rdt)

Goal: provide a **reliable channel** abstraction to the application on top of an *unreliable* network channel. Complexity depends on what the channel can do to packets (lose / corrupt / reorder). Neither side knows the other's state unless told via a message — "that's why we need a protocol." We develop the sender and receiver as **finite state machines (FSMs)**, considering unidirectional data (control info still flows both ways).

- **rdt 1.0:** perfectly reliable channel — just send and receive.
- **rdt 2.0:** channel may *flip bits*. Add a *checksum* to detect errors and **ACK/NAK** feedback: the sender retransmits on a NAK ("stop-and-wait"). *Fatal flaw:* what if the ACK/NAK itself is corrupted?
- **rdt 2.1:** add a 1-bit **sequence number** (0/1) so the receiver can discard duplicates; the sender retransmits if the ACK/NAK is corrupted. Two sequence numbers suffice for stop-and-wait.

- **rdt 2.2:** a **NAK-free** protocol — the receiver ACKs the last correctly received packet (including its seq #); a *duplicate ACK* means "retransmit." (TCP uses this idea.)
- **rdt 3.0:** channel may also *lose* packets. Add a **countdown timer**: if no ACK arrives in a "reasonable" time, retransmit. A delayed (not lost) packet causes a harmless duplicate that sequence numbers already handle.

3.4.1 Why stop-and-wait performs terribly

With a 1 Gbps link, 15 ms one-way prop. delay (so $RTT = 30$ ms), and $L = 8000$-bit packets: $d_{\text{trans}} = L/R = 8\,\mu$s. Sender utilization:

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} \approx 0.00027.$$

The protocol — not the link — limits performance.

3.5 Pipelined Protocols

Pipelining: allow multiple "in-flight," yet-to-be-ACKed packets. This needs a larger range of sequence numbers and buffering at sender and/or receiver. Three packets in flight improves utilization $3\times$:

$$U_{\text{sender}} = \frac{3\,L/R}{RTT + L/R}.$$

3.5.1 Go-Back-N (GBN)

- Sender keeps a window of up to N consecutive transmitted-but-unACKed packets, with a k-bit sequence number.
- **Cumulative ACK:** ACK(n) acknowledges *all* packets up to and including n; on receipt the window slides to begin at $n + 1$.
- A single timer covers the oldest in-flight packet; **timeout(n)** retransmits packet n and *all* higher-seq packets in the window.
- **Receiver** is simple: it only tracks the next expected seq (`rcv_base`), discards out-of-order packets, and re-ACKs the highest in-order seq received.

3.5.2 Selective Repeat (SR)

- The receiver *individually* ACKs each correctly received packet and *buffers* out-of-order packets for later in-order delivery.
- The sender keeps a (conceptual) timer *per* unACKed packet and on timeout retransmits *only* that one packet.
- **The dilemma:** sequence numbers wrap around. To avoid the receiver mistaking a retransmission for new data, the **window size must be** $\leq \frac{1}{2}$ the sequence-number space. (Receiver windows in $[\texttt{rcvbase}, \texttt{rcvbase} + N - 1]$ and $[\texttt{rcvbase} - N, \texttt{rcvbase} - 1]$ must not overlap.)

3.6 TCP: Connection-Oriented Transport (RFCs 793, 1122, 2018, 5681, 7323)

TCP is point-to-point, reliable, in-order **byte stream** (no message boundaries), **full-duplex**, connection-oriented (handshake first), flow-controlled, and congestion-controlled. It pipelines with **cumulative ACKs** and a window set by flow and congestion control. **MSS** is the maximum segment size (the payload), distinct from MTU.

3.6.1 Segment structure

Key fields: source/dest ports; 32-bit **sequence number** (byte-stream number of the first data byte — counting *bytes*, not segments); 32-bit **acknowledgement number** (next byte expected, cumulative); header length; flags (**ACK**, **RST**/**SYN**/ **FIN** for connection management, **C**/**E** for congestion notification); **receive window** (`rwnd`, for flow control); checksum; options.

3.6.2 Sequence numbers and ACKs

Telnet example: A types 'C'. A → B: `Seq=42, ACK=79, data='C'`. B echoes: `Seq=79, ACK=43, data='C'`. A acks the echo: `Seq=43, ACK=80`. How a receiver handles out-of-order segments is *not* specified by the standard — it's up to the implementer.

3.6.3 RTT estimation and timeout

The timeout must exceed the RTT, but RTT varies. `SampleRTT` is measured per segment (ignoring retransmissions) and smoothed with an **exponential weighted moving average (EWMA)**:

$$\text{EstimatedRTT} = (1-\alpha)\,\text{EstimatedRTT} + \alpha\,\text{SampleRTT}, \qquad \alpha = 0.125.$$

A deviation estimate adds a safety margin:

$$\text{DevRTT} = (1-\beta)\,\text{DevRTT} + \beta\,|\text{SampleRTT} - \text{EstimatedRTT}|, \qquad \beta = 0.25,$$

$$\boxed{\text{TimeoutInterval} = \text{EstimatedRTT} + 4\,\text{DevRTT}}.$$

3.6.4 Reliable transfer and fast retransmit

A single timer is kept for the oldest unACKed segment. On a **timeout**, TCP retransmits the segment that timed out and restarts the timer. A cumulative ACK can "cover" an earlier lost ACK.

Fast retransmit: a missing segment shows up as repeated cumulative ACKs for the same byte. On receiving **three duplicate ACKs** ("triple duplicate"), TCP resends the missing segment *immediately*, without waiting for the timeout. The receiver's ACK rules (RFC 5681) include *delayed ACKs* (wait up to 500 ms to piggyback) and immediate *duplicate ACKs* on out-of-order arrivals.

3.7 TCP Flow Control

Problem: the network layer may hand data to the receiver faster than the application reads it from the socket buffer. **Flow control** stops the sender from overflowing the *receiver*'s buffer:

- The receiver advertises free buffer space in the `rwnd` field of every segment.
- The sender limits its in-flight (unACKed) data to $\leq$ `rwnd`, guaranteeing the receive buffer never overflows.

Distinct from congestion control: flow control protects one *receiver*; congestion control protects the *network*.

3.8 TCP Connection Management

3.8.1 Why 2-way handshakes fail

A 2-way handshake can't reliably establish state because of variable delays, retransmitted/duplicated connection requests, and reordering — leading to **half-open connections** (server thinks it's connected, no client) or duplicate-data acceptance.

3.8.2 The 3-way handshake

1. Client $\rightarrow$ server: `SYN`, `Seq=x` (choose initial seq x).
2. Server $\rightarrow$ client: `SYNACK`, `Seq=y, ACK=x+1` (choose initial seq y, ack the SYN).
3. Client $\rightarrow$ server: `ACK, ACK=y+1` (may carry data).

States transition `LISTEN` $\rightarrow$ `SYN_SENT`/`SYN_RCVD` $\rightarrow$ `ESTAB`. ("On belay? / Belay on. / Climbing.")

```python
# Server
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 12000))
s.listen(1)
conn, addr = s.accept() # completes the 3-way handshake

# Client
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(('serverName', 12000)) # sends SYN, receives SYNACK, sends ACK
```

Listing 10: The socket calls that drive the TCP handshake

3.8.3 Closing a connection

Each side closes its own half by sending a segment with `FIN=1` and responding to a received FIN with an ACK (which may be combined with its own FIN). Simultaneous FIN exchanges are handled.

3.9 Principles of Congestion Control

Congestion: "too many sources sending too much data too fast for the network to handle." Manifestations: long queueing delays and packet loss (buffer overflow). A top-10 networking problem — and *different* from flow control.

- **Scenario 1** (two flows, one router, infinite buffers): as arrival rate $\to R/2$, delay $\to \infty$. Max per-connection throughput is $R/2$.
- **Scenario 2** (finite buffers, retransmissions): the transport-layer input $\lambda'_{in} \geq \lambda_{in}$ because of retransmissions. Costs of congestion: more work per delivered packet, and *un-needed duplicates* (from premature timeouts) waste capacity and lower achievable throughput.
- **Scenario 3** (multi-hop): when a packet is dropped *downstream*, all the upstream capacity and buffering already spent on it is wasted.

Two approaches:

- **End-to-end** (TCP): no explicit network feedback; congestion *inferred* from observed loss/delay.
- **Network-assisted:** routers give direct feedback (e.g. TCP ECN, ATM, DECbit), indicating congestion level or an explicit rate.

3.10 TCP Congestion Control

3.10.1 AIMD

Senders probe for bandwidth: **Additive Increase** (raise `cwnd` by 1 MSS every RTT while no loss) and **Multiplicative Decrease** (halve `cwnd` on a loss event) — the characteristic "sawtooth."

- **TCP Reno:** on triple-duplicate-ACK loss, cut to half.
- **TCP Tahoe:** on a *timeout*, cut all the way to 1 MSS.

AIMD is a distributed, asynchronous algorithm with good stability and network-wide fairness properties.

3.10.2 Sending rate and the window

The sender limits unACKed data to the congestion window:

$$\text{LastByteSent} - \text{LastByteAcked} \;\leq\; \texttt{cwnd}, \qquad \text{TCP rate} \approx \frac{\texttt{cwnd}}{RTT} \text{ bytes/sec.}$$

3.10.3 Slow start, congestion avoidance, fast recovery

- **Slow start:** begin with `cwnd`= 1 MSS and *double* `cwnd` every RTT (increment per ACK) — exponential ramp.
- **Switch to congestion avoidance** when `cwnd` reaches `ssthresh`; thereafter increase *linearly* (+1 MSS per RTT). On any loss, `ssthresh` is set to half the `cwnd` at the loss event.
- **Loss handling:** timeout → `ssthresh`= `cwnd`/2, `cwnd`= 1 MSS (back to slow start). Triple duplicate ACK → `ssthresh`=`cwnd`/2, `cwnd`=`ssthresh`+3 (**fast recovery**).

3.10.4 TCP throughput

Ignoring slow start, if loss occurs at window W, the window oscillates between $W/2$ and W, so the average window is $\frac{3}{4}W$:

$$\text{avg. throughput} \approx \frac{3}{4} \cdot \frac{W}{RTT}.$$

The **Mathis** formula relates throughput to loss probability L:

$$\text{throughput} \approx \frac{1.22 \cdot \text{MSS}}{RTT\sqrt{L}}.$$

For "long, fat pipes" (e.g. 1500-byte segments, $RTT = 100$ ms, target 10 Gbps) you'd need $\sim$83 333 in-flight segments and a tiny loss rate $L \approx 2 \times 10^{-10}$ — motivating high-speed TCP variants.

3.10.5 TCP CUBIC

Instead of pure linear increase, after halving the window on loss, ramp back toward $W_{\max}$ (the window at which loss was last seen) *quickly at first, then cautiously* as you near $W_{\max}$. Window growth is a *cubic* function of the time since the last loss (K is when `cwnd` would reach $W_{\max}$). CUBIC is the Linux default and the most popular TCP for busy web servers.

3.10.6 Delay-based congestion control (and BBR)

Goal: "keep the end-to-end pipe *just* full, but not fuller" — keep the bottleneck busy without inducing loss/large queues. Track $RTT_{\min}$ (uncongested) and compute uncongested throughput = $\texttt{cwnd}/RTT_{\min}$:

- measured throughput close to uncongested ⇒ increase `cwnd` linearly (path not congested);
- measured throughput far below ⇒ decrease `cwnd` (path congested).

This achieves high throughput with low delay and *without forcing loss.* **BBR** (a delay/bandwidth-based design) is deployed on Google's backbone.

3.10.7 ECN (Explicit Congestion Notification)

Network-assisted: a router marks two bits in the IP header (ToS field) to signal congestion; the destination sets the `ECE` bit on its ACK to notify the sender. Involves both IP (marking) and TCP (C/E header bits).

3.10.8 TCP fairness

If K sessions share a bottleneck of rate R, each "should" get R/K. TCP is fair *under idealized assumptions* (same RTT, fixed number of sessions, all in congestion avoidance): additive increase moves both flows along a 45° line, multiplicative decrease pulls them toward equal share. Caveats:

- **UDP** apps (some multimedia) ignore congestion control — there's no "Internet police."
- **Parallel connections:** an app opening many TCP connections grabs a larger share (on a link of rate R with 9 existing connections, a new app asking for 1 TCP gets $R/10$, but asking for 11 gets $R/2$).

4 The Network Layer: Data Plane

4.1 Overview: Data Plane vs. Control Plane

The network layer transports a segment from the sending to the receiving host: the sender **encapsulates** segments into **datagrams** and passes them to the link layer; the receiver delivers segments up to the transport layer. Crucially, the network layer runs in *every* Internet device — hosts *and* routers. A router examines header fields of all IP datagrams passing through it and moves each from an input port to the appropriate output port.

4.1.1 Two key functions

- **Forwarding** (*local*, per-router): move a packet from a router's input link to the correct output link. Analogy: getting through a *single* highway interchange.
- **Routing** (*global*): determine the end-to-end path packets take from source to destination, via routing algorithms. Analogy: planning the whole trip.

4.1.2 The two planes

- **Data plane:** local, per-router; decides how a datagram *arriving* on an input port is forwarded to an output port. Runs in *hardware* at nanosecond timescales.
- **Control plane:** network-wide logic; decides how a datagram is *routed* end-to-end. Runs in *software* at millisecond timescales. Two approaches:
  - **Traditional (per-router):** a routing-algorithm component in *every* router interacts to build forwarding tables.
  - **SDN (Software-Defined Networking):** a *remote* controller computes and installs forwarding tables into all routers.

4.2 Network-Layer Service Models

The design question: *what service model should the "channel" that carries datagrams offer?* Possible per-datagram guarantees include guaranteed delivery or guaranteed delivery within a delay bound (e.g. < 40 ms). Per-*flow* guarantees include in-order delivery, a minimum bandwidth, or bounded jitter (limits on inter-packet-spacing changes).

Architecture	Model	Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	CBR	constant rate	yes	yes	yes
ATM	ABR	guaranteed min	no	yes	no
Internet	Intserv Guar. (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

The Internet's **best-effort** model makes *no* guarantees on delivery, timing/order, or bandwidth — yet it succeeded because its simplicity enabled wide deployment, sufficient bandwidth provisioning makes real-time apps "good enough most of the time," replicated CDN/datacenter services sit close to clients, and congestion control of elastic traffic helps. "It's hard to argue with success."

4.3 Inside a Router

A generic router has **input ports**, a high-speed **switching fabric**, **output ports**, and a **routing processor**. The forwarding data plane (hardware, nanoseconds) is separate from the routing/management control plane (software, milliseconds). Analogy: a roundabout (fabric) with entry stations (input ports), exit roads (output ports), and a station manager (routing processor).

4.3.1 Input port functions

1. **Line termination** (physical layer): bit-level reception.
2. **Link-layer protocol** (e.g. Ethernet) receive.
3. **Lookup, forwarding, queueing:** *decentralized switching* — using header fields, look up the output port in a forwarding table held in *input-port* memory ("match plus action"), ideally at **line speed**. If datagrams arrive faster than the fabric can absorb them, **input queueing** occurs.

Destination-based forwarding uses only the destination IP address (traditional); **generalized** forwarding uses any set of header fields (the SDN "match+action" model).

4.4 Longest Prefix Matching (LPM)

When looking up a destination, a router uses the **longest address prefix** that matches. Example forwarding table ($*$ = don't-care):

Prefix	Meaning	Interface
`11001000 00010111 00010***`	.../21	0
`11001000 00010111 00011000`	.../24	1
`11001000 00010111 00011***`	.../21	2
otherwise	default	3

A datagram to `11001000 00010111 00011000 10101010` matches both the .../21 (interface 2) *and* the .../24 (interface 1) entry; LPM picks the **more specific** /24 $\Rightarrow$ **interface 1**. A datagram to `...00010110 ...` matches only the first prefix $\Rightarrow$ interface 0.

LPM is typically done in hardware with **TCAM** (Ternary Content-Addressable Memory): present an address and it returns the matching entry in *one clock cycle* regardless of table size (Cisco Catalyst: $\sim$1M entries).

4.5 Switching Fabrics

The fabric transfers a packet from an input link to an output link. The **switching rate** (often stated as a multiple of the line rate) should ideally be N times the line rate R for N ports (aggregate NR). Three classic fabric types:

- **Via memory (1st generation):** a traditional computer copies the packet into system memory under CPU control — *two* bus crossings per datagram; speed limited by memory bandwidth.

- **Via a bus:** the datagram crosses a shared bus directly; speed limited by **bus contention** (e.g. a 32 Gbps bus suffices for access routers).
- **Via an interconnection network:** e.g. a **crossbar** (originally built to connect multiprocessors); multiple packets can traverse simultaneously, overcoming the single-bus bottleneck.

4.6 Input Queueing and HOL Blocking

If the fabric is slower than the combined input rate, packets queue at the inputs (delay and loss from input-buffer overflow). Worse, **output-port contention** means only one packet destined to a given output can cross at a time; a blocked packet at the *front* of an input queue stalls everything behind it — even packets bound for idle outputs. This is **Head-of-Line (HOL) blocking**.

4.7 Output Queueing and Buffering

Buffering is required when datagrams arrive from the fabric faster than the output link transmits (rate R); overflow causes queueing delay and loss. Two policy questions arise: a **drop policy** (which datagram to drop when full) and a **scheduling discipline** (which queued datagram to send next).

4.7.1 How much buffering?

- **RFC 3439 rule of thumb:** average buffering ≈ typical RTT (∼250 ms) × link capacity C (e.g. 10 Gbps ⇒ 2.5 Gbit buffer).
- **More recent (with N flows):** buffering $\approx \dfrac{\text{RTT} \times C}{\sqrt{N}}$.
- **Bufferbloat:** *too much* buffering (common in home routers) inflates delay — long RTTs give poor real-time performance and sluggish TCP response. Recall delay-based congestion control: "keep the bottleneck just full enough, but no fuller."

4.8 Buffer Management and Packet Scheduling

4.8.1 Buffer management

- **Drop:** which packet to drop when the buffer is full — *tail drop* (drop the arriving packet) or *priority* drop (drop/remove by priority).
- **Marking:** which packets to *mark* to signal congestion — ECN, RED (Random Early Detection).

4.8.2 Scheduling policies

- **FCFS / FIFO:** transmit in order of arrival.
- **Priority:** classify arrivals into priority classes (using any header fields); always serve the highest-priority non-empty queue, FCFS within a class.
- **Round Robin (RR):** cyclically scan class queues, sending one packet from each in turn.
- **Weighted Fair Queueing (WFQ):** generalized RR; each class i has weight w_i and receives a share $\dfrac{w_i}{\sum_j w_j}$ of the link each cycle — giving a per-class **minimum bandwidth guarantee**. (Scheduling choices raise net-neutrality questions: who gets the best performance?)

4.9 IP: The Internet Protocol — Datagram Format

The host/router network layer comprises the **IP protocol** (datagram format, addressing, packet-handling conventions), the **ICMP protocol** (error reporting, router signaling), and forwarding tables built by routing protocols (OSPF, BGP) or an SDN controller.

Key IPv4 header fields:

- **Version**, **header length**, **type of service** (Diffserv bits 0–5, ECN bits 6–7).
- **Total length** (bytes; max 64 KB, typically ≤ 1500).
- **16-bit identifier**, **flags**, **fragment offset** — for fragmentation/reassembly.

- **TTL** (remaining hops, decremented at each router).
- **Upper-layer protocol** (e.g. TCP, UDP — used for demux at the destination).
- **Header checksum**; **source** and **destination** 32-bit IP addresses; **options** (rarely used).

Overhead: 20 bytes IP + 20 bytes TCP = 40 bytes per segment (before application data).

4.10 IP Fragmentation and Reassembly

Every link has an **MTU** (Maximum Transmission Unit) — the largest link-level frame it can carry — and different link types have different MTUs. A too-large IP datagram is **fragmented** within the network into several datagrams, and **reassembled only at the destination**. Header fields coordinate this:

- **Identifier** — same value on all fragments of one original datagram.
- **Flag (MF, "more fragments")** — 1 on all but the last fragment.
- **Fragment offset** — position of this fragment's data in the original, *counted in units of 8 bytes*.

4.10.1 Worked example

A 4000-byte datagram (20-byte header, 3980 bytes of data) over a link with MTU = 1500: each fragment carries at most $\lfloor (1500 - 20)/8 \rfloor \times 8 = 1480$ data bytes.

Fragment	length	ID	MF flag	offset	
1	1500	x	1	0	
2	1500	x	1	185	(1480/8)
3	1040	x	0	370	(2960/8)

```python
def fragment(total_len, mtu, header=20):
    data_left = total_len - header
    max_data = ((mtu - header) // 8) * 8 # payload/frag, multiple of 8
    frags, offset_bytes = [], 0
    while data_left > 0:
        take = min(max_data, data_left)
        more = 1 if data_left - take > 0 else 0
        frags.append({'length': take + header,
                      'offset': offset_bytes // 8, # counted in 8-byte units
                      'MF': more})
        offset_bytes += take
        data_left -= take
    return frags

for f in fragment(4000, 1500):
    print(f)
# {'length': 1500, 'offset': 0, 'MF': 1}
# {'length': 1500, 'offset': 185, 'MF': 1}
# {'length': 1040, 'offset': 370, 'MF': 0}
```

Listing 11: Computing IPv4 fragments (offset in 8-byte units)

4.11 IP Addressing, Subnets, and CIDR

An **IP address** is a 32-bit identifier for an **interface** (the connection between a host/router and a physical link). Routers usually have several interfaces; a host usually has one or two (wired Ethernet, WiFi). Addresses are written in **dotted-decimal** notation (e.g. `223.1.1.1` = `11011111 00000001 00000001 00000001`).

4.11.1 Subnets

A **subnet** is a set of interfaces that can reach each other *without* passing through an intervening router. Addresses have structure: a high-order **subnet part** (common to all devices in the subnet) and a low-order **host part**. *Recipe:* detach each interface from its host/router; each resulting isolated "island" is a subnet.

4.11.2 CIDR

CIDR (Classless Inter-Domain Routing, "cider") lets the subnet portion be *any* length: `a.b.c.d/x`, where x is the number of subnet-part bits (the **subnet mask**). A `/24` means the first 24 bits are the subnet part.

```python
import ipaddress

net = ipaddress.ip_network('200.23.16.0/23')
print(net.network_address) # 200.23.16.0
print(net.broadcast_address) # 200.23.17.255
print(net.netmask) # 255.255.254.0
print(net.num_addresses) # 512 = 2**(32-23)
print(net.num_addresses - 2) # 510 usable hosts (minus network+broadcast)

# Longest-prefix match: the most specific matching route wins
routes = [ipaddress.ip_network(c) for c in ('200.23.16.0/20',
                                            '200.23.18.0/23')]
dst = ipaddress.ip_address('200.23.18.55')
best = max((r for r in routes if dst in r), key=lambda r: r.prefixlen)
print(best) # 200.23.18.0/23 (the /23 beats the /20)
```

Listing 12: Subnet math and longest-prefix match with the ipaddress module

4.12 Getting an Address: DHCP

Two sub-questions: how a *host* gets its host part, and how a *network* gets its subnet part.

4.12.1 Host part — DHCP

A host's address is either hard-coded by a sysadmin or, more commonly, obtained dynamically via **DHCP** (Dynamic Host Configuration Protocol) — "plug and play." DHCP lets a host obtain an address when it joins a network, renew a lease, reuse freed addresses, and support mobile users. The four-message exchange (**DORA**), all broadcast, is:

1. **Discover** — "is there a DHCP server out there?" (src `0.0.0.0:68`, dst `255.255.255.255:67`).
2. **Offer** — server proposes an address (`yiaddr`) and lease time.
3. **Request** — client asks for the offered address.
4. **ACK** — server confirms.

The first two steps can be skipped if a client is reusing a previously allocated address (RFC 2131). DHCP returns more than an address: the **first-hop (default) router**, the **DNS server**, and the **subnet mask**. The whole DHCP message is carried in UDP → IP → Ethernet and broadcast on the LAN.

4.12.2 Network part — ISPs and ICANN

A network gets its subnet part as an allocated block of its provider ISP's address space. **ICANN** (via 5 Regional Registries) allocates IP blocks and manages the DNS root zone and TLD delegation. The last IPv4 blocks were handed to the registries in 2011 — motivating NAT and IPv6.

4.13 Hierarchical Addressing and Route Aggregation

Hierarchical allocation lets an ISP advertise a *single* aggregate route ("send me anything beginning `200.23.16.0/20`") covering many customer organizations — keeping global routing tables small. If one organization moves to a different ISP, that ISP advertises a **more specific** route (e.g. `200.23.18.0/23`); LPM ensures the more specific route wins, so traffic still reaches the moved organization.

4.14 NAT: Network Address Translation

All devices in a local network share *one* public IPv4 address as far as the outside world is concerned. Internally they use a **private** block (`10.0.0.0/8`, `172.16.0.0/12`, or `192.168.0.0/16`) that is only meaningful locally.

Advantages: only one ISP-provided address is needed; local addresses can change without notifying the outside; the ISP can change without renumbering local hosts; and inside devices aren't directly addressable from outside (a security side-effect).

4.14.1 How it works

The NAT router transparently rewrites addresses using a **translation table**:

- **Outgoing:** replace (source IP, port) with (NAT IP, *new* port), and record the mapping.
- **Incoming:** replace the destination (NAT IP, new port) with the stored (source IP, port).

WAN side (NAT IP, port)	LAN side (host IP, port)
`138.76.29.7, 5001`	`10.0.0.1, 3345`

Controversy: NAT makes a network-layer device manipulate port numbers (a transport-layer concept), violating the end-to-end argument, and complicates **NAT traversal** (reaching a server *behind* a NAT). Purists argue address shortage "should" be solved by IPv6 — but NAT is here to stay (home, institutional, and 4G/5G networks).

4.15 IPv6

Motivation: the 32-bit IPv4 space was being exhausted; secondary goals were faster processing (a fixed 40-byte header) and better handling of *flows*.

4.15.1 Datagram format

Fields: version, **priority**, **flow label** (identifies datagrams in the same flow — concept loosely defined), payload length, **next header** (upper-layer protocol), **hop limit**, and **128-bit** source/destination addresses. Compared with IPv4, IPv6 *drops*:

- the header **checksum** (speeds router processing),
- in-network **fragmentation/reassembly**,
- **options** (moved to an upper-layer "next-header" chain).

4.15.2 Transition: tunneling

Not all routers can be upgraded at once (no "flag days"), so IPv4 and IPv6 must coexist. **Tunneling** carries an IPv6 datagram as the *payload* of an IPv4 datagram between IPv6-capable routers separated by IPv4-only infrastructure (a "packet within a packet"). The entry router (dual-stack IPv6/v4) encapsulates the IPv6 datagram in an IPv4 datagram addressed to the far tunnel end; the exit router decapsulates and forwards the original IPv6 datagram. The inner IPv6 source/destination (e.g. A→F) are preserved; the outer IPv4 header uses the tunnel endpoints (e.g. B→E). Tunneling is used widely elsewhere too (4G/5G).

Adoption is slow (25+ years): ∼40% of Google's clients reach it over IPv6 (2023), and ∼1/3 of US government domains are IPv6-capable.

5 The Network Layer: Control Plane

5.1 Overview: Two Approaches

The control plane is the network-wide logic that computes the forwarding tables the data plane uses. Recall the split: **forwarding** moves a packet from a router's input to the right output (data plane, local, per router); **routing** determines the end-to-end route packets take (control plane). Two ways to structure it:

- **Per-router control (traditional):** a routing-algorithm component runs in *every* router; the components interact (exchange messages) and each router computes its own forwarding table.
- **Logically centralized control (SDN):** a remote controller computes forwarding tables and installs them into the routers, which become simple match+action devices.

Internet instantiations: OSPF and BGP (per-router); OpenFlow with ODL/ONOS controllers (SDN); ICMP for control messaging; SNMP and NETCONF/YANG for network management.

5.2 Routing Protocols and the Graph Abstraction

Goal: find "good" (least-cost / fastest / least-congested) paths from source to destination hosts through the router network — a top-10 networking problem. Model the network as a graph $G = (N, E)$:

- N = set of routers, e.g. $\{u, v, w, x, y, z\}$; E = set of links.
- $c_{a,b}$ = cost of the direct link (a, b); $c_{a,b} = \infty$ if a, b are not direct neighbors. The operator defines cost: always 1 (hop count), or inversely related to bandwidth or to congestion.
- Path cost = sum of link costs; we want the **least-cost path**.

Classification. *Global* information (link-state): every router knows the complete topology and all link costs. *Decentralized* (distance-vector): an iterative process where each router initially knows only the costs to its attached neighbors and exchanges info with them. Orthogonally, routes are *static* (change slowly) or *dynamic* (periodic updates / react to cost changes).

5.3 Link-State Routing: Dijkstra's Algorithm

Every node floods its link costs network-wide (a **link-state broadcast**), so all nodes learn the identical full topology. Dijkstra computes least-cost paths from one *source* to all other nodes (yielding that source's forwarding table). It is iterative: after k iterations, the least costs to k destinations are final. Notation: $D(v)$ = current least-cost estimate to v; $p(v)$ = predecessor of v on that path; N' = set of finalized nodes.

```
Initialization:
  N' = {u}
  for all nodes v:
     if v adjacent to u: D(v) = c(u,v) # direct-neighbor cost only
     else: D(v) = infinity

Loop until all nodes in N':
  find w not in N' with minimum D(w); add w to N'
  update D(v) for all v adjacent to w and not in N':
     D(v) = min( D(v), D(w) + c(w,v) )
```

Listing 13: Dijkstra's link-state algorithm (source u)

5.3.1 Worked example (source u)

Costs: $c_{u,v}$=2, $c_{u,x}$=1, $c_{u,w}$=5, $c_{v,x}$=2, $c_{v,w}$=3,
$c_{x,w}$=3, $c_{x,y}$=1, $c_{w,y}$=1, $c_{w,z}$=5, $c_{y,z}$=2.

Step	N'	$D(v), p$	$D(w), p$	$D(x), p$	$D(y), p$	$D(z), p$
0	u	2,u	5,u	**1**,u	∞	∞
1	ux	**2**,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	$uxyv$		**3**,y			4,y
4	$uxyvw$					**4**,y
5	$uxyvwz$					

Resulting least costs from u: x=1, v=2, y=2 (u-x-y), w=3 (u-x-y-w), z=4 (u-x-y-z). The forwarding table sends v out link (u, v) and everything else out (u, x).

```
import heapq

def dijkstra(graph, src): # graph[u] = {v: cost, ...}
    dist = {n: float('inf') for n in graph}
    prev = {n: None for n in graph}
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: # stale entry
```

```
            continue
        for v, w in graph[u].items():
            if d + w < dist[v]:
                dist[v], prev[v] = d + w, u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev
```

Listing 14: Dijkstra's least-cost paths

Complexity. n iterations, each checking the remaining nodes $\Rightarrow n(n+1)/2$ comparisons $= O(n^2)$; $O(n \log n)$ with a heap. **Message complexity:** each router broadcasts its link state (each broadcast crosses $O(n)$ links) $\Rightarrow O(n^2)$ overall. **Oscillations** are possible when link costs depend on carried traffic (routes "slosh" between paths).

5.4 Distance-Vector Routing: Bellman-Ford

Based on the **Bellman-Ford equation** (dynamic programming). Let $D_x(y)$ be the least cost from x to y:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \},$$

the minimum over all neighbors v of (x-to-v direct cost) + (v's least cost to y). The neighbor v achieving the minimum is the **next hop**.

5.4.1 Worked example

Reaching z from u with neighbors v, x, w, where $D_v(z)$=5, $D_x(z)$=3, $D_w(z)$=3 and $c_{u,v}$=2, $c_{u,x}$=1, $c_{u,w}$=5:

$$D_u(z) = \min\{\, 2{+}5,\ 1{+}3,\ 5{+}3 \,\} = \min\{7, 4, 8\} = 4,$$

so the next hop toward z is x.

The algorithm is iterative, asynchronous, distributed, and self-stopping. Each node keeps its own distance vector plus its neighbors' vectors. A local iteration is triggered by a *local link-cost change* or a *DV update from a neighbor*; the node recomputes with Bellman-Ford, and *only if its DV changed* does it notify its neighbors — who then repeat, only if necessary. No message, no action. Under natural conditions the estimates $D_x(y)$ converge to the true least costs $d_x(y)$.

```
def dv_update(x, link_cost, neighbor_dv, destinations):
    # link_cost[v] = cost of the direct x->v link (v a neighbor)
    # neighbor_dv[v][y] = neighbor v's estimated cost to y
    Dx = {}
    for y in destinations:
        Dx[y] = min(link_cost[v] + neighbor_dv[v][y] for v in link_cost)
    return Dx # notify neighbors iff Dx changed
```

Listing 15: A distance-vector update at node x (one Bellman-Ford step)

5.4.2 Convergence example (3 nodes)

Costs $c_{x,y}$=2, $c_{x,z}$=7, $c_{y,z}$=1. From its neighbors' vectors, node x computes

$$D_x(y) = \min\{c_{x,y}{+}D_y(y),\ c_{x,z}{+}D_z(y)\} = \min\{2{+}0,\ 7{+}1\} = 2,$$

$$D_x(z) = \min\{c_{x,y}{+}D_y(z),\ c_{x,z}{+}D_z(z)\} = \min\{2{+}1,\ 7{+}0\} = 3.$$

After exchanging vectors, all three settle (here $D_x = [0, 2, 3]$). Information diffuses one hop per round: a node's state at t=0 can influence computations up to k hops away by t=k.

5.4.3 Link-cost changes: good news vs. bad news

- **Good news travels fast:** when a link cost *drops*, a node updates its DV and informs neighbors; the news settles in a few rounds.
- **Bad news travels slowly — count-to-infinity:** suppose y's link to x jumps to 60 while z still advertises a cost-5 path. y computes "6 via z" and tells z; z then computes "7 via y" and tells y; $y \to 8$; $z \to 9$; ... incrementing slowly toward the true cost.

- **Poisoned reverse:** if x routes to a destination via z, x advertises its distance to that destination as ∞ to z, so z won't loop back through x. Fixes 2-node loops, not larger ones.

	Link-State (LS)	**Distance-Vector (DV)**
Knowledge	Full topology (flooded)	Neighbors + their vectors
Algorithm	Dijkstra	Bellman-Ford
Messages	$O(n^2)$; each floods its link state	Only between neighbors; varies
Convergence	$O(n^2)$; possible oscillations	Varies; routing loops, count-to-infinity
Robustness	Router can advertise wrong *own* cost; each computes its own table	A router can advertise wrong *path* cost ("low cost to everywhere" = black-holing); errors *propagate*

5.5 Making Routing Scalable: Autonomous Systems

The idealized "all routers identical, flat network" does not scale: billions of destinations can't fit in one table, and exchanging them would swamp links. Also, each network's admin wants **autonomy** over its own routing. So routers are aggregated into **Autonomous Systems (ASes)** / domains:

- **Intra-AS (intra-domain):** routing among routers in the same AS; all its routers run the same intra-AS protocol.
- **Inter-AS (inter-domain):** routing between ASes, done by **gateway routers** at the AS edge (which also run intra-AS routing).

Intra-AS routing fills table entries for internal destinations; inter-AS plus intra-AS together fill entries for external destinations. When a router in AS1 receives a datagram for outside AS1, inter-domain routing must (1) learn which destinations are reachable through AS2 vs. AS3 and (2) propagate that reachability to all AS1 routers (so a packet is sent to the right gateway).

5.6 Intra-AS Routing: RIP, EIGRP, OSPF

- **RIP** (RFC 1723): classic distance-vector, DVs exchanged every 30 s; no longer widely used.
- **EIGRP:** DV-based, long Cisco-proprietary (opened in 2013, RFC 7868).
- **OSPF** (Open Shortest Path First, RFC 2328): the dominant IGP; IS-IS (an ISO standard) is essentially equivalent.

OSPF is "open" (publicly specified) and a classic *link-state* protocol: each router floods link-state advertisements to all others in the AS *directly over IP* (not TCP/UDP), builds the full topology, and runs Dijkstra to compute its forwarding table. Cost metrics are configurable (bandwidth, delay). All OSPF messages are **authenticated** to prevent malicious intrusion.

Hierarchical OSPF uses two levels — local *areas* and a *backbone*. LSAs are flooded only within an area (or the backbone); each node knows its area's detailed topology and only the *direction* to other areas.

- **Area border routers** summarize their area's distances and advertise them into the backbone.
- **Backbone routers** run OSPF limited to the backbone.
- **Boundary routers** connect to other ASes.

5.7 Inter-AS Routing: BGP

BGP (Border Gateway Protocol) is the de-facto inter-domain routing protocol — "the glue that holds the Internet together." It lets a subnet advertise its existence and the destinations it can reach: "I am here, here's who I can reach, and how." BGP gives each AS a way to:

- obtain destination reachability from neighboring ASes (**eBGP**);
- determine routes based on reachability *and policy*;
- propagate reachability to all AS-internal routers (**iBGP**);
- advertise reachability to neighboring networks.

Gateway routers run *both* eBGP (to other ASes) and iBGP (inside the AS).

5.7.1 BGP sessions and messages

Two BGP peers exchange messages over a semi-permanent **TCP** connection. BGP is a **path-vector** protocol: advertisements carry *paths* to destination prefixes. Messages (RFC 4271): `OPEN` (open + authenticate the peer), `UPDATE` (advertise a new path or withdraw an old one), `KEEPALIVE` (keep the session alive; also ACKs OPEN), `NOTIFICATION` (report errors / close).

5.7.2 Path attributes

An advertised route = **prefix** + **attributes**. Two key attributes:

- **AS-PATH:** the list of ASes the advertisement has traversed (also used to detect and avoid loops).
- **NEXT-HOP:** the specific internal-AS router leading toward the next-hop AS.

Policy-based routing: an *import* policy accepts/declines a received path (e.g. "never route through AS Y"); an *export* policy decides whether to advertise a path onward. When AS3's gateway advertises $\langle AS3, X\rangle$ to AS2, AS3 *promises* to forward datagrams toward X.

5.7.3 Path propagation and forwarding tables

AS2 router 2c receives $\langle AS3, X\rangle$ via eBGP from 3a, propagates $\langle AS2, AS3, X\rangle$ inside AS2 via iBGP, and 2a advertises it to AS1 via eBGP. AS1's gateway 1c may learn *two* paths to X ($\langle AS2, AS3, X\rangle$ and $\langle AS3, X\rangle$), pick one by policy, and flood it via iBGP. To build the actual forwarding table, a router combines BGP ("X is reachable via gateway 1c") with intra-AS OSPF ("to reach 1c, use this interface"): X-bound traffic leaves on the interface OSPF says leads to that gateway.

5.7.4 Hot-potato routing and policy

Hot-potato routing: among multiple gateways that can reach X, pick the one with least *intra-domain* cost — get the packet out of your own AS as soon as possible, regardless of inter-domain hop count. **Policy example:** an ISP typically routes traffic only to/from its *customer* networks and refuses to carry *transit* traffic between other ISPs (no revenue). So a provider B that learns a path may choose *not* to advertise it if none of the endpoints are B's customers, and a dual-homed customer x won't advertise a route that would make it transit between two providers.

Route selection order (when multiple routes to a destination exist):

1. highest **local-preference** (a policy decision);
2. shortest **AS-PATH**;
3. closest **NEXT-HOP** (hot-potato routing);
4. additional tie-breakers.

5.7.5 Why intra- and inter-AS routing differ

Policy: inter-AS admins want control over who routes through them (policy dominates); intra-AS is a single admin, so policy matters less. **Scale:** hierarchy shrinks table size and update traffic. **Performance:** intra-AS can optimize for performance; inter-AS is dominated by policy.

5.8 The SDN Control Plane

SDN separates a *logically centralized* controller from simple data-plane switches:

- **Generalized forwarding:** switches hold **flow tables** of **match+action** rules (**OpenFlow**); a rule matches header fields across layers 2–4 and acts (forward to port(s), drop, modify a field, or send to the controller).
- **Controller:** maintains network state and offers a **southbound** API (e.g. OpenFlow) to program switches and a **northbound** API to control apps (routing, access control, load balancing). Production controllers include OpenDaylight (ODL) and ONOS.

Benefits: programmability, one place for network-wide logic, and open interfaces (no vendor lock-in).

5.9 ICMP: Internet Control Message Protocol

Hosts and routers use **ICMP** to exchange network-level information — error reporting (unreachable host / network / port / protocol) and echo request/reply (used by `ping`). ICMP sits "above" IP but its messages are carried *in* IP datagrams; each message has a **type**, a **code**, plus the first 8 bytes of the IP datagram that caused the error.

Type	Code	Description	Type	Code	Description
0	0	echo reply (ping)	3	6	dest. network unknown
3	0	dest. network unreachable	3	7	dest. host unknown
3	1	dest. host unreachable	8	0	echo request (ping)
3	2	dest. protocol unreachable	9	0	route advertisement
3	3	dest. port unreachable	10	0	router discovery
4	0	source quench (unused)	11	0	TTL expired
			12	0	bad IP header

5.9.1 Traceroute via ICMP

The source sends sets of UDP segments with increasing TTL (first set TTL= 1, next TTL= 2, ...). The datagram in the n-th set expires at router n, which discards it and returns an ICMP **TTL-expired** (type 11, code 0), often naming the router; the source records the RTT. It *stops* when a segment finally reaches the destination host, which returns **port unreachable** (type 3, code 3).

5.10 Network Management: SNMP and NETCONF/YANG (Brief)

A **managing server** monitors and controls **managed devices**, each running an **agent** that exposes a **MIB** (Management Information Base — the data objects). **SNMP** carries request/response (`GET`/`SET`) messages and unsolicited **trap** messages (agent→server event reports). **NETCONF** (with the **YANG** data-modeling language) is the modern approach to configuring and managing devices.

6 The Link Layer and LANs

6.1 Introduction and Services

Terminology: **nodes** = hosts and routers; **links** connect adjacent nodes (wired, wireless, LANs); a link-layer (Layer-2) packet is a **frame** that encapsulates a datagram. The link layer's job is to move a datagram from one node to a *physically adjacent* node over a single link. A datagram may traverse different link protocols on different links (WiFi on the first, Ethernet on the next), each offering different services. *Transportation analogy:* a trip Princeton→Lausanne (limo, plane, train) — the tourist is the datagram, each leg is a link, each mode is a link-layer protocol, and the travel agent is the routing algorithm.

Possible services:

- **Framing / link access:** encapsulate the datagram (header + trailer); arbitrate access if the medium is shared; put source/destination **MAC addresses** (distinct from IP addresses) in the header.
- **Reliable delivery** between adjacent nodes — seldom used on low-error wired links, valuable on high-error wireless links. (Why both link-level and end-to-end reliability? To avoid costly end-to-end retransmissions across a single lossy link.)
- **Flow control**, **error detection**, **error correction**, and **half-/full-duplex** operation.

The link layer lives in the **network adapter (NIC)** — on-chip or a card implementing the link and physical layers — a combination of hardware, software, and firmware attached to the host's system bus. Sending side: encapsulate, add error-check bits. Receiving side: check for errors, extract the datagram, pass it up.

6.2 Error Detection and Correction

The sender appends **EDC** (error-detection-and-correction) bits to the data D. Detection is *not* 100% reliable (some errors slip through undetected, though rarely); a larger EDC field improves detection/correction.

- **Single-bit parity:** one bit makes the number of 1s even (or odd); detects an *odd* number of bit errors only.

- **Two-dimensional parity:** parity per row *and* per column; *detects and corrects* any single-bit error by locating its row and column — forward error correction, no retransmission.
- **Internet checksum:** the one's-complement sum from the transport layer — software, relatively weak.
- **CRC (Cyclic Redundancy Check):** stronger, hardware-friendly, used by Ethernet and 802.11 WiFi.

6.2.1 CRC

Given data D and a standard $(r{+}1)$-bit generator G, the sender finds the r-bit remainder

$$R = \text{remainder}\left(\frac{D \cdot 2^r}{G}\right) \pmod 2,$$

so that $D \cdot 2^r \oplus R$ is exactly divisible by G (mod 2); it transmits $D \cdot 2^r \oplus R$. The receiver divides the received bits by G; a *nonzero* remainder means an error. CRC detects all burst errors of length $\leq r$. **Example:** $D = 101110$, $G = 1001$ ($r = 3$); mod-2 dividing 101110000 by 1001 gives $R = 011$, so the transmitted frame is 101110 011.

```python
def crc_remainder(data_bits, gen_bits):
    G = int(gen_bits, 2)
    r = len(gen_bits) - 1
    reg = int(data_bits, 2) << r # append r zeros: D * 2^r
    for i in range(len(data_bits) + r - 1, r - 1, -1):
        if reg & (1 << i): # subtract (XOR) G aligned at bit i
            reg ^= G << (i - r)
    return format(reg, f'0{r}b')

print(crc_remainder('101110', '1001')) # -> '011'
```

Listing 16: Mod-2 (CRC) remainder

6.3 Multiple Access Protocols

Two link types: **point-to-point** (one sender/receiver, e.g. switch-to-host, PPP dial-up) and **broadcast** (a shared medium: old Ethernet, upstream cable HFC, 802.11 WiFi, 4G/5G, satellite). On a broadcast link, two or more simultaneous transmissions **collide** (a node receiving two signals at once). A multiple-access (MAC) protocol is a *distributed* algorithm that decides when a node may transmit — and coordination must use the shared channel itself (no separate out-of-band channel).

Ideal MAC on a rate-R channel: (1) one active node gets the full rate R; (2) M active nodes each get $\approx R/M$; (3) fully decentralized (no coordinator, no clock sync); (4) simple. Three families:

6.3.1 Channel partitioning

- **TDMA** (time-division): each station gets a fixed slot per round (slot = one frame time); unused slots go idle (6-station LAN with only 1,3,4 active ⇒ slots 2,5,6 wasted).
- **FDMA** (frequency-division): each station gets a fixed frequency band; idle bands wasted.

Fair and collision-free, but inefficient at low load (a node is capped at R/N even when it is the only one active).

6.3.2 Random access

No coordination; collisions happen and must be recovered from.

- **Slotted ALOHA:** synchronized slots (one frame each); a node with a fresh frame transmits in the next slot; on collision it retransmits in each later slot with probability p until success. *Pros:* a lone node uses the full rate; highly decentralized; simple. *Cons:* collisions and idle slots waste slots; needs clock sync.
- **Pure ALOHA:** unsynchronized (simpler, no slots) but more collisions.
- **CSMA** (carrier sense): *listen before transmit* — send only if the channel is idle. Collisions still occur because of propagation delay (two nodes sense idle and both start); the whole colliding frame time is wasted.
- **CSMA/CD** (collision detection): abort as soon as a collision is heard, cutting wasted time. Used by classic **Ethernet**. Detection is easy on wire, hard on wireless.

- **CSMA/CA** (collision avoidance): used by WiFi (802.11), where a node cannot detect collisions while transmitting.

Slotted-ALOHA efficiency. With N nodes each transmitting in a slot with probability p,

$$P(\text{a given node succeeds}) = p(1-p)^{N-1}, \qquad P(\text{some node succeeds}) = Np(1-p)^{N-1}.$$

Maximizing over p and letting $N \to \infty$ gives max efficiency $1/e \approx 0.37$ — at best, the channel does useful work 37% of the time (pure ALOHA: $1/(2e) \approx 0.18$).

```
1. Adapter gets a datagram from the network layer, creates a frame.
2. Sense the channel: if idle, start transmitting; if busy, wait until idle.
3. Whole frame sent with no collision -> done.
4. Collision detected while sending -> abort, send a jam signal.
5. Binary exponential backoff: after the m-th collision, pick K uniformly
   from {0, 1, ..., 2^m - 1}; wait K * 512 bit-times; go to step 2.
   (More collisions -> larger backoff window.)
```

Listing 17: Ethernet CSMA/CD with binary exponential backoff

6.3.3 Taking turns

- **Polling:** a master node invites each node to transmit in turn (polling overhead, latency, single point of failure; Bluetooth uses polling).
- **Token passing:** a control token is passed node-to-node; only its holder may transmit (token overhead/latency, token loss; FDDI, token ring).

"Taking turns" aims for the best of both worlds — efficient at high load (like partitioning) yet low-delay at low load (like random access).

6.4 MAC Addressing and ARP

6.4.1 MAC addresses

An interface's **MAC (LAN / physical / Ethernet) address** is 48 bits (most LANs), burned into the NIC ROM (sometimes software-settable), written in hex (e.g. `1A-2F-BB-76-09-AD`). Its job: get a frame from one interface to another *physically-connected* interface on the same subnet. Allocation is administered by **IEEE** (manufacturers buy address blocks for uniqueness). Analogy: **MAC = Social-Security number** (flat, portable — moves with the adapter), **IP = postal address** (hierarchical, depends on the attached subnet, *not* portable).

6.4.2 ARP

ARP (Address Resolution Protocol) answers: knowing an interface's IP address on my LAN, what is its MAC address? Each node keeps an **ARP table** of ⟨IP, MAC, TTL⟩ entries (TTL typically ∼20 min). When A wants to send to B but lacks B's MAC:

1. A **broadcasts** an ARP query (destination MAC `FF-FF-FF-FF-FF-FF`) containing B's IP address; all LAN nodes receive it.
2. B recognizes its own IP and **replies (unicast)** to A with its MAC address.
3. A caches the ⟨IP, MAC⟩ mapping (with a TTL) in its ARP table.

6.4.3 Routing to another subnet

To reach a host B on *another* subnet via first-hop router R: A knows B's IP, R's IP, and (via ARP) R's MAC. The IP source/destination stay A/B end-to-end; only the *link-layer* addresses change hop by hop.

Hop	IP src → dst	MAC src → dst	Comment
$A \to R$	$A \to B$	$\text{MAC}_A \to \text{MAC}_R^{(\text{left})}$	Frame carries A-to-B datagram; dest MAC is R's
$R \to B$	$A \to B$	$\text{MAC}_R^{(\text{right})} \to \text{MAC}_B$	R removes/re-adds Layer-2 header; dest MAC is B's

Key insight: routers rewrite the Layer-2 header at each hop, while the Layer-3 (IP) destination remains the true endpoint B throughout.

6.5 Ethernet

The dominant, cheap, simple wired LAN, keeping up with the speed race (10 Mbps–400 Gbps). The physical topology evolved from a **bus** (all nodes in one collision domain, popular through the mid-90s) to today's **switched** star: an active Layer-2 switch at the center, each spoke its own collision domain, so nodes do not collide.

6.5.1 Frame structure

Field	Size	Purpose
Preamble	8 B	7 bytes `10101010` + 1 byte `10101011`; sync receiver clock
Dest. MAC	6 B	If it matches (or is broadcast) pass up to the network layer, else discard
Source MAC	6 B	Sending adapter's address
Type	2 B	Upper-layer protocol (mostly IP) — for demux at the receiver
Data (payload)	46–1500 B	The IP datagram
CRC	4 B	Error detection; frame dropped if it fails

Ethernet is **connectionless** (no handshake between NICs) and **unreliable** (the receiving NIC sends no ACK/NAK — data in dropped frames is recovered only if a higher layer such as TCP retransmits). Its MAC protocol is **unslotted CSMA/CD with binary backoff**. The **802.3** family shares one MAC protocol and frame format across many physical media/speeds (100BASE-TX copper, 100BASE-FX fiber, ...).

6.6 Switches

A **switch** is an active link-layer device: it *stores and forwards* Ethernet frames, examining each incoming frame's MAC address to selectively forward it to one or more outgoing links. It is **transparent** (hosts are unaware of it) and **plug-and-play / self-learning** (no configuration). Because each host has a dedicated full-duplex link to the switch, there are no collisions and **multiple transmissions can happen simultaneously** — e.g. A→A′ and B→B′ at once (but not two frames to the same output A′). Each link is its own collision domain.

6.6.1 Switch table and self-learning

The switch keeps a **switch table** of ⟨MAC address, interface, timestamp⟩ (it resembles a routing table but is built differently). It *learns* by recording the *source* MAC and arriving interface of each frame. Forwarding logic:

```
When a frame arrives on an interface:
  1. record (source MAC, arriving interface) in the switch table
  2. look up the destination MAC in the switch table
  3. if an entry is found:
       if it maps to the arriving interface: drop (filter)
       else: forward on that one interface
     else:
       flood (forward on all interfaces except the arriving one)
```

Listing 18: Switch frame filtering / forwarding

Interconnected switches self-learn in exactly the same way, so a frame from A to a distant G is forwarded switch-to-switch toward G once the relevant locations have been learned.

6.6.2 Switches vs. routers

Both are store-and-forward. A **switch** is a Layer-2 device: it examines link-layer headers and builds its table by *flooding + learning* on MAC addresses. A **router** is a Layer-3 device: it examines network-layer headers and computes tables with *routing algorithms* on IP addresses, and it isolates broadcast domains.

6.7 VLANs

Motivation. A single switched LAN is one broadcast domain: all Layer-2 broadcast traffic (ARP, DHCP, unknown-MAC floods) must cross the entire LAN (scaling, security, and privacy issues), and moving a user's office can force an awkward physical/logical mismatch (a CS user physically on the EE switch but wanting to stay logically on CS).

Port-based VLANs. A VLAN-capable switch is configured so its ports are grouped into isolated virtual LANs — one physical switch behaves as several virtual switches (e.g. ports 1–8 = EE, ports 9–15 = CS). **Traffic isolation:**

frames to/from ports 1–8 reach only ports 1–8. **Dynamic membership:** ports can be reassigned among VLANs. **Inter-VLAN traffic** goes through a router (just as with separate switches; vendors sell combined switch+router boxes).

VLANs across switches — trunking. A **trunk port** carries frames of *multiple* VLANs between switches. Such frames need a VLAN ID, added by **802.1Q**: a 4-byte tag is inserted after the source-MAC field — a 2-byte Tag Protocol Identifier (`0x8100`) plus a 2-byte Tag Control field (a 12-bit VLAN ID and a 3-bit priority, like IP ToS) — and the CRC is recomputed.

6.8 Link Virtualization and Data Centers (Brief)

MPLS (Multi-Protocol Label Switching) forwards on short fixed-length *labels* rather than IP longest-prefix match, enabling faster forwarding and traffic engineering. **Data-center networks** interconnect tens of thousands of hosts using hierarchical topologies (access / aggregation / core tiers, often with rich multipath) for high bisection bandwidth.

6.9 A Day in the Life of a Web Request (Synthesis)

Fetching `www.google.com` exercises the whole stack at once:

1. **DHCP** (UDP/IP/Ethernet, broadcast): the laptop obtains its IP address, subnet mask, first-hop router, and DNS server.
2. **ARP:** to reach the DNS server / first-hop router, the host ARPs for the router's MAC.
3. **DNS** (UDP): resolve the name (root → TLD → authoritative, via the local resolver).
4. **TCP** 3-way handshake to the web server (port 80/443).
5. **HTTP** GET; the response returns down the stack (HTTP→TCP→IP→Ethernet), through switches (self-learning, Layer-2) and routers (LPM forwarding, Layer-2 rewriting) along the way.

7 Cheat Sheet

7.1 Key Formulas

Quantity	**Formula**
Transmission delay	$d_{\text{trans}} = L/R \quad (L \text{ bits, rate } R \text{ bps})$
Propagation delay	$d_{\text{prop}} = d/s \quad (s \approx 2 \times 10^8 \text{ m/s})$
Nodal delay	$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$
Traffic intensity	$La/R \quad (< 1 \text{ good}, \to 1 \text{ large delay}, > 1 \text{ unbounded})$
End-to-end throughput	$\min(R_s, R_c)$; shared: $\min(R_s, R_c, R/N)$
Packet-switch capacity (binomial)	$P(X{=}k) = \binom{n}{k} p^k (1-p)^{n-k}$
Non-persistent HTTP (per object)	$2\,RTT + T_{\text{transmit}}$
File dist. (client–server)	$D_{\text{c-s}} \geq \max\{NF/u_s,\ F/d_{\min}\}$
File dist. (P2P)	$D_{\text{P2P}} \geq \max\{F/u_s,\ F/d_{\min},\ NF/(u_s + \sum u_i)\}$
EWMA RTT	$\text{EstRTT} = (1{-}\alpha)\text{EstRTT} + \alpha\,\text{SampleRTT}$, α=0.125
RTT deviation	$\text{DevRTT} = (1{-}\beta)\text{DevRTT} + \beta\lvert\text{SampleRTT} - \text{EstRTT}\rvert$, β=0.25
TCP timeout	$\text{TimeoutInterval} = \text{EstRTT} + 4\,\text{DevRTT}$
Stop-and-wait utilization	$U = \dfrac{L/R}{RTT + L/R}$
Pipelined (n packets)	$U = \dfrac{n\,L/R}{RTT + L/R}$
TCP rate	$\approx \texttt{cwnd}/RTT$
TCP avg. throughput	$\frac{3}{4}\,W/RTT$
TCP throughput vs. loss (Mathis)	$1.22 \cdot \text{MSS} \,/\, (RTT\sqrt{L})$
SR window constraint	window size $\leq \frac{1}{2}\times$ seq-number space
Router switching rate (ideal)	$N \times R \quad (N \text{ ports, line rate } R)$
Router buffer (RFC 3439)	$\text{RTT} \times C$
Router buffer (N flows)	$\text{RTT} \times C/\sqrt{N}$
WFQ per-class share	$w_i / \sum_j w_j$
Hosts in a `/x` subnet	$2^{32-x} - 2 \quad$ (minus network + broadcast)
Fragment offset	(bytes before this fragment) / 8
Max frag. payload	$\lfloor(\text{MTU} - 20)/8\rfloor \times 8$ bytes
Bellman-Ford eqn.	$d_x(y) = \min_v\{\, c(x,v) + d_v(y)\,\}$
Dijkstra complexity	$O(n^2)$, or $O(n \log n)$ with a heap
Slotted ALOHA (max eff.)	$1/e \approx 0.37$
Pure ALOHA (max eff.)	$1/(2e) \approx 0.18$
CSMA/CD efficiency	$\approx \dfrac{1}{1 + 5\,t_{\text{prop}}/t_{\text{trans}}}$
CRC remainder	$R = \text{remainder}(D{\cdot}2^r/G) \mod 2 \quad (r{+}1\text{-bit } G)$

7.2 Well-Known Ports

Port	Protocol	Transport / notes
20/21	FTP (data/control)	TCP
22	SSH	TCP
25	SMTP	TCP (mail *between* servers)
53	DNS	UDP (TCP for large transfers)
80	HTTP	TCP
143	IMAP	TCP (mail retrieval)
161	SNMP	UDP
443	HTTPS / HTTP/3	TCP (TLS) / UDP (QUIC)

7.3 HTTP Methods and Status Codes

Method	Purpose	Code	Meaning
GET	Retrieve object	200	OK
POST	Send form data in body	301	Moved Permanently
HEAD	Headers only	304	Not Modified (conditional GET)
PUT	Upload / replace object	400	Bad Request
		404	Not Found
		505	HTTP Version Not Supported

7.4 DNS Record Types

Type	Maps / does
A	name → IPv4 address
AAAA	name → IPv6 address
NS	delegate a domain to a name server
CNAME	alias → canonical name
MX	domain → mail server (lower priority number tried first)

7.5 TCP vs. UDP

	TCP	UDP
Connection	Yes (3-way handshake)	No
Reliability	Reliable, in-order byte stream	Best effort, may lose/reorder
Flow control	Yes (`rwnd`)	No
Congestion control	Yes (AIMD / CUBIC / BBR)	No
Header	20+ bytes	8 bytes
Demux key	4-tuple (src/dst IP + port)	2-tuple (dst IP + port)
Typical use	Web, email, file transfer	DNS, SNMP, streaming, HTTP/3

7.6 Circuit vs. Packet Switching

	Circuit	**Packet**
Resources	Reserved (dedicated)	Shared on demand
Setup	Required	None
Performance	Guaranteed	Best effort (delay/loss)
Efficiency	Idle circuits waste capacity	Statistical multiplexing
Used by	Traditional telephone net	The Internet

7.7 Go-Back-N vs. Selective Repeat

	Go-Back-N	**Selective Repeat**
ACK type	Cumulative	Individual
Receiver buffering	None (discard out-of-order)	Buffers out-of-order
Timers	One (oldest packet)	One per unACKed packet
On loss	Resend window from n onward	Resend only the lost packet

7.8 The Five-Layer Stack (with units and example protocols)

Layer	**Unit**	**Examples**
5. Application	message	HTTP, SMTP, IMAP, DNS
4. Transport	segment	TCP, UDP
3. Network	datagram	IP (the "narrow waist"), routing protocols
2. Link	frame	Ethernet, 802.11 (WiFi), PPP
1. Physical	bit	copper, fiber, radio

7.9 TCP Congestion-Control State Reactions

Event	**Action**
Connection start	`cwnd`= 1 MSS; slow start (double per RTT)
`cwnd` $\geq$ `ssthresh`	Enter congestion avoidance (+1 MSS / RTT)
Triple duplicate ACK (Reno)	`ssthresh`=`cwnd`/2; `cwnd`=`ssthresh`+3 (fast recovery)
Timeout (Tahoe)	`ssthresh`=`cwnd`/2; `cwnd`= 1 MSS; slow start

7.10 Router Data Plane vs. Control Plane

	Data plane	**Control plane**
Scope	Local, per-router	Network-wide
Job	Forwarding (input $\rightarrow$ output port)	Routing (build tables)
Implemented in	Hardware	Software
Timescale	Nanoseconds	Milliseconds
Approaches	Match+action lookup	Per-router alg. *or* SDN controller

7.11 Packet Scheduling Policies

Policy	Rule	Property
FCFS / FIFO	Serve in arrival order	Simple; no isolation
Priority	Highest non-empty class first (FCFS within)	Low-priority starvation risk
Round Robin	One packet per class, cyclically	Fair-ish, no weights
WFQ	Class i gets $w_i / \sum_j w_j$ share	Per-class min-bandwidth guarantee

7.12 IPv4 Header Fields (Quick Reference)

Field	Purpose
Version / Header len	IP version; header size in 32-bit words
Type of Service	Diffserv (bits 0–5) + ECN (bits 6–7)
Total length	Datagram size in bytes (max 64 KB)
Identifier / Flags / Offset	Fragmentation and reassembly
TTL	Max remaining hops (decremented per router)
Upper-layer protocol	Demux key (6 = TCP, 17 = UDP)
Header checksum	Error check over the header only
Source / Dest address	32-bit endpoints

7.13 CIDR / Subnet Mask Quick Reference

Prefix	Mask	Addresses	Usable hosts ($2^{32-x}-2$)
/8	255.0.0.0	2^{24}	16,777,214
/16	255.255.0.0	2^{16}	65,534
/20	255.255.240.0	2^{12}	4,094
/23	255.255.254.0	2^{9}	510
/24	255.255.255.0	2^{8}	254
/30	255.255.255.252	2^{2}	2 (point-to-point links)

7.14 Private and Special IPv4 Addresses

Block / address	Use
`10.0.0.0/8`	Private (RFC 1918)
`172.16.0.0/12`	Private (RFC 1918)
`192.168.0.0/16`	Private (RFC 1918)
`127.0.0.0/8`	Loopback
`0.0.0.0`	“This host” / unspecified (DHCP source)
`255.255.255.255`	Limited broadcast (DHCP destination)
Ports 67 / 68	DHCP server / client (over UDP)

7.15 IPv4 vs. IPv6

	IPv4	**IPv6**
Address size	32-bit	128-bit
Header	Variable ($\geq$ 20 B)	Fixed 40 B
Checksum	Yes	No (speeds forwarding)
Fragmentation	Routers *and* host	Host only (not in-network)
Options	In header	"Next-header" chain
Notation	Dotted decimal	Colon-hex
Address setup	DHCP / static	SLAAC / DHCPv6

7.16 DHCP (DORA) and NAT

Step / concept	**Meaning**
Discover	Client broadcasts to find a server
Offer	Server proposes address (`yiaddr`) + lease
Request	Client asks for the offered address
ACK	Server confirms; also returns router, DNS, mask
NAT out	(src IP, port) $\rightarrow$ (public IP, new port), record in table
NAT in	(public IP, new port) $\rightarrow$ stored (src IP, port)

7.17 Routing Algorithms and Protocols

	Link-State	**Distance-Vector**
Knows	Full topology	Neighbors + their vectors
Computes with	Dijkstra	Bellman-Ford
Talks to	All (flooding)	Neighbors only
Weakness	Oscillations	Count-to-infinity
Example	OSPF	RIP

	Intra-AS (IGP)	**Inter-AS (BGP)**
Scope	Within one AS	Between ASes
Type	OSPF (LS), RIP (DV)	Path-vector
Goal	Least cost	Policy (economics)
Key attrs	Link costs, areas	AS-PATH, NEXT-HOP, local-pref
Selection	Shortest path	Local-pref > short AS-PATH > hot-potato

7.18 ICMP Common Messages

Type	**Name**	**Used by / meaning**
8 / 0	Echo request/reply	`ping`
3	Destination unreachable	network/host/port/frag-needed (codes)
11	TTL expired	`traceroute`

7.19 Multiple Access Protocols

Family	Protocols	Idea
Partitioning	TDMA, FDMA	Fixed slots/bands; no collisions, low-load waste
Random access	ALOHA, CSMA/CD, CSMA/CA	Contend; detect/avoid & recover from collisions
Taking turns	Polling, token passing	Coordinate whose turn it is

7.20 Error Detection Methods

Method	Strength
1-bit parity	Detects odd # of bit errors
2-D parity	Detects + *corrects* a single-bit error
Internet checksum	Weak; software (transport layer)
CRC	Strong; hardware; detects bursts $\leq r$ bits (Ethernet, WiFi)

7.21 Link Layer: MAC vs. IP, Switch vs. Router

	MAC address	IP address
Size	48-bit	32-bit (v4)
Structure	Flat	Hierarchical
Scope	One LAN	End-to-end
Assignment	Burned into NIC (portable)	Subnet-dependent (DHCP/static)
Resolved by	—	ARP (IP $\rightarrow$ MAC on a LAN)

	Switch (L2)	Router (L3)
Uses	MAC addresses	IP addresses
Table built by	Self-learning (source MAC)	Routing algorithms
Unknown dest	Flood	Drop / default route
Broadcast domain	Does *not* isolate	Isolates
Config	Plug-and-play	Needs addressing/routing

7.22 Ethernet Frame Fields

Field	Size	Purpose
Preamble	8 B	Clock synchronization
Dest / Source MAC	6 B each	Link-layer addresses
Type	2 B	Upper-layer demux (usually IP)
Data (payload)	46–1500 B	The IP datagram
CRC	4 B	Error detection (frame dropped on failure)

7.23 Intra-AS Routing Protocols

Protocol	Type	Notes
RIP	Distance-vector	DVs every 30 s; mostly historical
EIGRP	Distance-vector	Cisco, opened 2013
OSPF / IS-IS	Link-state (Dijkstra)	Floods LSAs over IP; authenticated; areas

7.24 BGP: Attributes and Route Selection

Concept	Meaning
eBGP / iBGP	Learn reachability from other ASes / propagate it inside the AS
Transport	Semi-permanent TCP session between peers
AS-PATH	List of ASes the advert crossed (loop detection)
NEXT-HOP	Internal router toward the next-hop AS
Messages	OPEN, UPDATE, KEEPALIVE, NOTIFICATION

Route selection order: (1) highest local-preference (policy); (2) shortest AS-PATH; (3) closest NEXT-HOP (hot-potato); (4) other tie-breakers. **Hot-potato:** pick the gateway with least *intra*-domain cost.

7.25 ARP and Switch Self-Learning

Step / rule	Action
ARP query	Broadcast (dest MAC `FF-FF-FF-FF-FF-FF`) with target IP
ARP reply	Owner unicasts its MAC; requester caches ⟨IP,MAC,TTL⟩ (∼20 min)
Switch learn	Record (source MAC, arriving interface)
Dest known, same port	Drop (filter)
Dest known, other port	Forward on that one interface
Dest unknown / broadcast	Flood (all ports except arriving)

7.26 VLANs and 802.1Q

Concept	Meaning
Port-based VLAN	One physical switch acts as several virtual switches
Traffic isolation	Frames stay within their VLAN's ports
Inter-VLAN	Requires a router (Layer 3)
Trunk port	Carries multiple VLANs between switches
802.1Q tag	4 B: TPID `0x8100` + 12-bit VLAN ID + 3-bit priority; CRC recomputed

These notes paraphrase and reorganize the lecture material (Kurose & Ross, 8th ed.; UC Berkeley CS 168) into a single study reference spanning the full exam scope — Transport (Ch. 3), Network-layer Data Plane (Ch. 4) and Control Plane (Ch. 5), and the Link Layer (Ch. 6) — with added code (sockets, SMTP, checksum, fragmentation, subnets, Dijkstra, distance-vector, CRC, CSMA/CD, switch learning) and an expanded cheat sheet.